Human brain sensing using photonics.

Bradley Gondolosi

Supervisor: Haobo Li

School of Science and Engineering

University of Dundee

April 2025

Abstract

Accurate, non-invasive monitoring of cerebral blood flow is essential for advancing functional neuroimaging and understanding neural dynamics under cognitive stimulus. Traditional imaging methods are limited by cost, invasiveness, and poor temporal resolution, creating a need for more accessible, real-time alternatives.

This project aimed to evaluate whether laser speckle imaging, in combination with machine learning techniques, could classify brain states, relaxing versus visually stimulated, based on hemodynamic fluctuations in the occipital lobe. The study also explored the viability of convolutional neural networks on a small neuroimaging dataset, despite their known data dependency.

Raw LSI data was processed using three signal extraction techniques: Optical Flow, Cross Correlation, and Contrast Correlation. These one-dimensional time series were classified using Support Vector Machine, Random Forest, and Convolutional Neural Network architectures, ResNet-18 and VGG19. Dimensionality reduction via Principal Component Analysis and Relief-F optimised traditional classifiers. Convolutional Neural Networks were trained on image-based inputs from time-series plots and spectrograms.

Contrast Correlation paired with Support Vector Machine under an RBF kernel achieved the highest accuracy at 94.60%, while Random Forest using the same data reached 89.19%. In contrast, Convolutional Neural Networks underperformed (≤65.38%), showing poor generalisation due to dataset size. These findings confirm the discriminative strength of contrast-based temporal features and highlight the limitations of deep learning models on low-volume neuroimaging data.

This study demonstrates that using pre-processed LSI data, lightweight machine learning classifiers can accurately infer cognitive brain states. It lays the foundation for non-invasive, remote neuroimaging systems, with future applications in cognitive monitoring, brain-computer interfaces, and neurodiagnostics.

DECLARATION

I, Bradley Gondolosi, student of the School of Science and Engineering at the University of Dundee, hereby declare that I am the sole author of this thesis titled ‘Human brain sensing using photonics’. Unless otherwise specified, all work is my own and has not been previously accepted for a higher degree.

Signed:

Date: 21/04/25

(Bradley Gondolosi)

ACKNOWLEDGEMENTS

The author would like to express their sincerest gratitude to their supervisor, Dr Haobo Li, for giving up a generous portion of his time and for his continued guidance, support and encouragement throughout the project. His expertise and insightful feedback about the field of Machine Learning have been invaluable to developing and completing this thesis.

Finally, the author wishes to thank his mother for their encouragement, motivation, patience and understanding throughout this investigation.

CONTENTS

ABSTRACT ..... **I**

CONTENTS ..... **IV**

LIST OF TABLES ..... **VIII**

LIST OF FIGURES ..... **IX**

LIST OF EQUATIONS ..... **XIII**

MATHEMATICAL SYMBOLS AND NOTATION ..... **XVI**

CHAPTER 1 INTRODUCTION **1**

1.1 AIMS 2

1.2 OBJECTIVES 2

CHAPTER 2 LITERATURE REVIEW **3**

2.1 LASER SPECKLE IMAGING 3

2.2 DATA PROCESSING 5

2.2.1 Optical Flow *5*

2.2.3 Cross Correlation *7*

2.2.4 Contrast Correlation *8*

2.3 ARTIFICIAL INTELLIGENCE 9

2.3.1 Neural Network *11*

2.3.2 Supervised vs unsupervised training *13*

2.3.2 Support Vector Machine *14*

2.3.3 Random Forest *18*

2.4 CONFUSION MATRICES 19

2.5 KEY RELATED STUDIES IN NON-INVASIVE BRAIN ACTIVITY MONITORING 22

2.5.1 SUMMARY OF KEY RELATED STUDIES IN NON-INVASIVE BRAIN ACTIVITY MONITORING 28

CHAPTER 3 METHODOLOGY 31

3.1 PARTICIPANTS 31

3.2 DATA ACQUISITION 31

3.3 DATA PROCESSING 31

3.3.1 Optical Flow.....32

3.3.2 Cross-correlation 35

3.3.3 Contrast correlation.....36

3.4 DATA PROCESSING FOR TRADITIONAL MACHINE LEARNING ALGORITHMS VS CONVOLUTIONAL NEURAL NETWORK 37

3.5 TRADITIONAL MACHINE LEARNING ALGORITHM 38

3.5.1 Support Vector Machine (SVM).....39

3.5.2 Random Forest (RF).....40

3.6 CONVOLUTION NEURAL NETWORK (CNN) 41

3.6.1 VGG19.....42

3.6.2 ResNet-18.....42

CHAPTER 4 PRESENTING FINDINGS AND RESULTS 43

4.1 PROCESSING TECHNIQUES.....43

4.1.1 Optical Flow Farneback.....43

4.1.2 CNN input example images.....46

4.2 CLASSIFIER RESULTS 48

4.2.1 How to read the Confusion Matrices 49

4.2.2 Support Vector Machine *50*

4.2.3 Random Forest *52*

4.2.4 VGG19 ..... *53*

4.2.5 ResNet-18 ..... *54*

4.3 SUMMARY OF CLASSIFIER PERFORMANCES WITH PREPROCESSING TECHNIQUES 55

CHAPTER 5 COMPARATIVE ANALYSIS AND DISCUSSION 58

5.1 SIGNAL TRENDS IN PROCESSED OUTPUTS 58

5.2 CLASSIFIER COMPARISONS 58

5.2.1 Interpreting Confusion Matrices Beyond Accuracy *58*

5.2.2 Support Vector Machine (SVM) *60*

5.2.3 Random Forest (RF) *60*

5.2.4 VGG19 ..... *60*

5.2.5 ResNet-18 ..... *61*

5.3 FUTURE WORK 62

5.4 IMPLICATIONS FOR ARTIFICIAL INTELLIGENCE IN MEDICAL IMAGING 63

CHAPTER 6 CONCLUSION 65

CHAPTER 7 REFERENCES 66

CHAPTER 8 APPENDIX A CODE 70

SVM (OPTICAL/CROSS/CONTRAST) 70

RANDOM FOREST 76

IMAGE PLOT FOR QUIVER 82

X AND Y OPTICAL FLOW 83

IMAGE GENERATION 84

Optical flow *84*

Cross *86*

Contrast *88*

VGG19 90

RESNET-18 92

LIST OF TABLES

Table 2.1: Detailing different optical techniques with their respective strength, limitations and best applications(5–8) 3

Table 2.2: Labelling of a Confusion Matrix 20

Table 2.3: Details the results of models from the paper (38). 24

Table 2.4: Details the results of the EEG model from the paper(38). 25

Table 2.5: Summary of Key Related Studies in Non-Invasive Brain Activity Monitoring 28

Table 3.1: Labelling for a singular trial/HDF5 file 32

Table 4.1: Labelling of a Confusion Matrix within the context of this study 59

LIST OF FIGURES

Figure 2.1: Illustrates the difference in displacement between two images mathematically(12) 5

Figure 2.2: Example cross-correlation measurements (15)..... 8

Figure 2.3: Machine Learning is a type of Artificial Intelligence. Deep Learning is an especially complex part of Machine Learning. Source:(18) 10

Figure 2.4: Deep Learning algorithms don't need an engineer to identify features, but are capable of automatic feature engineering through their neural network. Source:(19).... 10

Figure 2.5: A simple artificial neural network 11

Figure 2.6: Regression task vs classification task source(28)..... 14

Figure 2.7: SVM plane. source(30)..... 15

Figure 2.8: A Hyperplane within a Support Vector Machine boundary (29) 15

Figure 2.9: Kernel types in SVM. source(32) 18

Figure 2.10: Illustrates experimental set-up with functional near-infrared spectroscopy where a laser beam illuminates the scalp whilst the subject is made to perform tasks that will induce haemodynamic responses in the visual cortex (occipital lobe). 23

Figure 2.11: Experiment set up from the paper (38). 24

Figure 2.12: (45) Mental state images captured during stimulus provided to the user to segregate EEG data for three different psychological states: (**a**) normal state, (**b**) relaxed state, and (**c**) stressed state. Images are captured using the Emotiv BrainViz software version 1.1. 28

Figure 2.13: (45) Command mapping for EEG BCI-controlled wheelchair..... 28

Figure 3.1: CNN options in MATLAB(50)..... 41

Figure 4.1: Optical flow of the sixth volunteer's laser speckle image pixels in x-position

showing amplitude of flow against data acquisition time (seconds).....43

Figure 4.2: Optical flow of the sixth volunteer's laser speckle image pixels in y-position showing amplitude of flow against data acquisition time (seconds).....43

Figure 4.3 Optical flow quiver plots displaying pixel brightness for successive frames during resting period on the top row and visual stimulus bottom row with vector map displaying pixel movement between frames. Obtained from Volunteer 3's 3rd trial.....45

Figure: 4.4 Both images use optical flow processing with output as line plots. Both images are from Volunteer 3's 1st Trial. Left is a 15 second rest segment. Right is the 15 second segment under visual stimulus directly after the rest period.....47

Figure: 4.5 Both images use optical flow processing with output in spectrogram format. Both images are from Volunteer 3's 1st Trial. Left is a 15 second rest segment. Right is the 15 second segment under visual stimulus directly after the rest period.....47

Figure: 4.6 Both images use Cross Correlation with output in line plot format. Both images are from Volunteer 3's 1st Trial. Left is a 15 second rest segment. Right is the 15 second segment under visual stimulus directly after the rest period.....48

Figure: 4.7 Both images use Contrast Correlation with output in line plot format. Both images are from Volunteer 3's 1st Trial. Left is a 15 second relaxing segment. Right is 15 second segment under stimulus.....48

Figure 4.8: Confusion matrix for Optical Flow processing and Support Vector Machine classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy: 78.38% 50

Figure 4.9: Confusion matrix for Cross Correlation processing and Support Vector Machine classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy 67.57%.....51

Figure 4.10: Confusion matrix for Contrast Correlation processing and Support Vector Machine classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy: 94.60%. 51

Figure 4.11: Confusion matrix for Optical Flow processing and Random Forest classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy 70.27% 52

Figure 4.12: Confusion matrix for Cross Correlation processing and Random Forest classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy 62.16%. 52

Figure 4.13: Confusion matrix for Contrast Correlation processing and Random Forest classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy 89.19%. 52

Figure 4.14: Confusion matrix for Optical Flow Line Plot 2D images and VGG19 classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy 42.31%. 53

Figure 4.15: Confusion matrix for Optical Flow Spectrogram 2D images and VGG19 classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy 50.00%. 53

Figure 4.16: Confusion matrix for Cross Correlation Line Plot 2D images and VGG19 classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy 50.00%. 53

Figure 4.17: Confusion matrix for Contrast Correlation Line Plot 2D images and VGG19 classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy 50.00%. 54

Figure 4.18: Confusion matrix for Optical Flow Line Plot 2D images and ResNet18 classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy 65.38% 54

Figure 4.19: Confusion matrix for Optical Flow Line Plot 2D images and ResNet18 classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy 53.85% 54

Figure 4.20: Confusion matrix for Cross Correlation Line Plot 2D images and ResNet18

classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy 61.54% 55

Figure 4.21: Confusion matrix for Contrast Correlation Line Plot 2D images and ResNet18 classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy 61.54%..... 55

Figure 5.1: VGG19 Training Curve when paired with contrast correlation images. 61

Figure 5.2: ResNet18 Training Curve when paired with contrast correlation images. ... 62

LIST OF EQUATIONS

Equation 2.1: The brightness constancy constraint equation represents pixel intensity invariance between two consecutive frames in optical flow analysis. 5

Equation 2.2: Optical flow constraint equation derived via Taylor series approximation, establishing a linear relationship between spatial and temporal image gradients and flow velocities (u, v). 6

Equation 2.3: Quadratic polynomial used in Farnebäck's dense optical flow method to approximate pixel neighbourhoods in motion estimation. 7

Equation 2.4: General continuous-time cross-correlation function measuring similarity between two time-shifted signals. 7

Equation 2.5 Pixel-wise zero-mean and unit-variance normalisation applied to each frame to ensure consistent contrast metrics. 9

Equation 2.6: Weighted sum input to a neuron in a neural network, where weights represent feature importance. 12

Equation 2.7: Simple binary step activation function determining node activation in a feedforward network. 12

Equation 2.8: General form of a hyperplane in an n-dimensional feature space, where 16

Equation 2.9: Marginal constraints defining the region separating class +1 and class -1. 16

Equation 2.10: Perpendicular distance from a point to the hyperplane in Euclidean space. 17

Equation 2.11: Margin width between the two supporting hyperplanes. Maximising this margin improves model generalisation. 17

Equation 2.12: Hard-margin SVM constraint ensuring all data points lie outside or on the margin. 17

Equation 2.13 Mean Squared Error used by Random Forest in regression tasks to optimise node splitting. 19

Equation 2.14 Gini Impurity Index used for evaluating the homogeneity of splits in classification trees. 19

Equation 2.15: Accuracy formula calculating the proportion of total correct classifications in a binary classification task. 20

Equation 2.16: Precision metric representing the proportion of predicted positives that are actually correct. 21

Equation 2.17: Recall (Sensitivity) metric indicating the model's ability to correctly identify positive cases. 21

Equation 2.18: Specificity metric quantifying the proportion of true negatives among all actual negative cases. 21

Equation 2.19 : Negative Predictive Value measuring the reliability of a negative prediction. 22

Equation 2.20: Prevalence metric expressing the proportion of actual positive cases in the dataset. 22

Equation 3.1: Normalises pixel intensity values to the range [0,1] across all frames to ensure numerical stability during flow estimation. 33

Equation 3.2: Represents local pixel neighbourhoods using a second-order polynomial 33

Equation 3.3: Describes how the change in gradient vectors between frames relates to pixel displacement using the polynomial model. 33

Equation 3.4: Converts flow magnitude and direction into a complex number for each pixel to facilitate summation. 34

Equation 3.5: Sums complex flow vectors over the image to estimate global motion in each frame. 34

Equation 3.6: Accumulates real and imaginary components over time to generate 1D motion trajectories. 34

Equation 3.7: Calculates the magnitude of motion at each time point, forming a final scalar signal input for machine learning. 35

Equation 3.8:Zero-mean normalisation of each frame by subtracting its average pixel intensity. 35

Equation 3.9:Computes the similarity between two consecutive normalised frames using 2D cross-correlation. 35

Equation 3.10: Extracts the peak value from the correlation matrix to represent temporal similarity between frames. 35

Equation 3.11:Standardises each frame by adjusting pixel values to have zero mean and unit variance. 36

Equation 3.12:Computes the similarity of contrast patterns between consecutive frames using dot product correlation. 37

MATHEMATICAL SYMBOLS AND NOTATION

Symbol	Meaning
$\in$	"is an element of" – denotes set membership
$\mathbb{R}$	The set of all real numbers
$\mathbb{R}^{m \times n}$	The set of all real-valued matrices with dimensions $m \times n$
∂	Partial derivative, used in gradient calculations
∇	Gradient operator. Represents the vector of partial derivatives
Δ	Change in value of ( )
$\sum$	Summation – used to sum a series of terms
$\cdot$	Dot product – scalar product of two vectors
$\hat{x}$	Unit vector or normalised version of x
x^T	Transpose of vector or matrix x
$\angle$	Phase angle of a complex number or signal
exp	Exponential function
j	Imaginary unit $j^2 = -1$

$\approx$	Approximately equal to
$=$	Equal to

CHAPTER 1 INTRODUCTION

In the field of neuroimaging in recent years, there have been many advancements in brain mapping to understand and view the different complexities within it. Advances in neuroimaging technologies have revolutionised the study of brain function, allowing researchers to map neural activity, quantify cerebral blood flow (CBF).

This study aims to show the significance of cerebral blood flow and brain state with the use of different preprocessing techniques and selective classifiers, focusing on how visual stimuli influence haemodynamic activity. Blood flow is recorded with a SWIR (short-wave infrared) laser paired with a high frames per second camera directed at the nape of the participant, the anatomical location of the occipital lobe. The acquired frames are analysed using multiple data processing techniques to compare their effectiveness in quantifying CBF and classifying brain states.

Optical flow approach captures the motion of brightness patterns across consecutive frames, providing temporal insights into CBF dynamics. Contrast correlation quantifies variations in blood flow intensity within individual frames by calculating the ratio of standard deviation to mean intensity, identifying regions with high or low blood flow. Cross-correlation, on the other hand, measures the similarity between successive frames to track the degree of change and the magnitude of haemodynamic shifts. These methods are compared to assess their computational efficiency, robustness, and accuracy in analysing cerebral blood flow. Finally, machine learning algorithms are applied to the processed data from each method to classify brain states, providing an evaluation of how the choice of data processing technique impacts the model's performance and accuracy.

This study benefits the future of neuroimaging by providing a verifiable standard of work that shows neural activity and functional brain mapping can be explored remotely with signal processing software rather than more expensive equipment on location with the patient. Combining Laser speckle imaging (LSI) with machine learning demonstrates another perspective to neuroimaging, as unlike traditional neuroimaging techniques, LSI offers real-time, non-invasive monitoring of cerebral blood flow (CBF) with high spatial

and temporal resolution, making it particularly valuable for studying fast, dynamic processes such as neurovascular coupling.

1.1 Aims

- Literature review of laser speckle imaging.
- Retrieve the human brain signal from the raw speckle data.
- Build AI models based on the processed data to distinguish two different brain states.
- Try different data processing techniques and machine learning classifiers.

1.2 Objectives

- Examine different studies to see the gap for my experiment.
- Examine the terms around 'Artificial Intelligence', specifically ones pertinent to this study.
- Process data with optical flow, cross-correlation and contrast correlation with detailed mathematics behind each of these systems.
- Use processed data on various machine learning algorithms.
- Detail the effectiveness of each combination of data processing technique and machine learning algorithm.
- Detailing technique(s) for Cerebral Blood Flow detection.

CHAPTER 2 LITERATURE REVIEW

2.1 Laser Speckle Imaging

Laser speckle imaging (LSI) is a widely used technique for measuring cerebral blood flow (CBF) and other physiological parameters. The technique works by analysing the speckle patterns generated by coherent light scattering from biological tissue. Its foundation was laid in 1981 when Fercher and Briers introduced Laser Speckle Contrast Analysis (LASCA), enabling blood flow quantification by measuring the spatial contrast of time-integrated speckle patterns in laser and camera. The technologies used in these imaging and analysis have significantly enhanced the resolution and sensitivity of LSI, making it a preferred tool in neuroimaging(1).

Laser speckle principle that light scattering from moving particles, such as red blood cells, produces dynamic speckle patterns. Analysing these patterns over time, it is possible to quantify the velocity and direction of blood flow. The technique's ability to capture events in real time makes it viable for studying fast physiological processes such as neurovascular coupling, where changes in neural activity that is linked to changes in CBF (2,3).

The occipital lobe function in the brain is the to act map the visual world, identify stimuli and ‘receive raw visual data from perceptual sensors in the eyes' retina’ making it the most viable area to identify neural activity as this correlates to cerebral blood flow in this area (4).

Table 2.1: Detailing different optical techniques with their respective strength, limitations and best applications(5–8)

Technique	Wavelength Range (nm)	Penetration Depth (cm)	Strengths	Limitations	Best Applications
Green Light	500-570	~0.1-0.3	High sensitivity for surface imaging	Limited to superficial tissue; high absorption by melanin	Superficial cortical imaging, vascular studies
NIR	650-900	~1-2	Deeper penetration, high Signal to Noise Ratio	Lower spatial resolution and vulnerability to motion	Cortical brain imaging, haemodynamic monitoring
SWIR	1000-2500	>2	Deep penetration; high Signal to Noise Ratio	High cost; complex setup	Deep brain and subcortical imaging
fNIRS	650-900	~1-3	Portable, real-time monitoring	Limited depth sensitivity; contamination by extracerebral signals	Cognitive neuroscience, developmental studies
DOT	650-900	~1-4	3D imaging: high spatial resolution	Complex setup; time-consuming computations	Volumetric brain imaging, deep brain activity mapping

2.2 Data Processing

The term data processing was first used in the 1950s as manual data processing. The term data processing can be assigned to mundane tasks with no connection to computer science, like recording transactions made. The invention of Structured Query Language in 1974 is a significant step that standardised data processing in relation to computer science and data analysis. Commercially available in 1979(9)SQL key functions are 'Data definition ' and ' Data retrieval. Data manipulation, Access control, Data sharing'(10).

2.2.1 Optical Flow

Used in image processing, optical flow is the measurement of correlation between two consecutive frames caused by the movement of object(s) within frames(11).

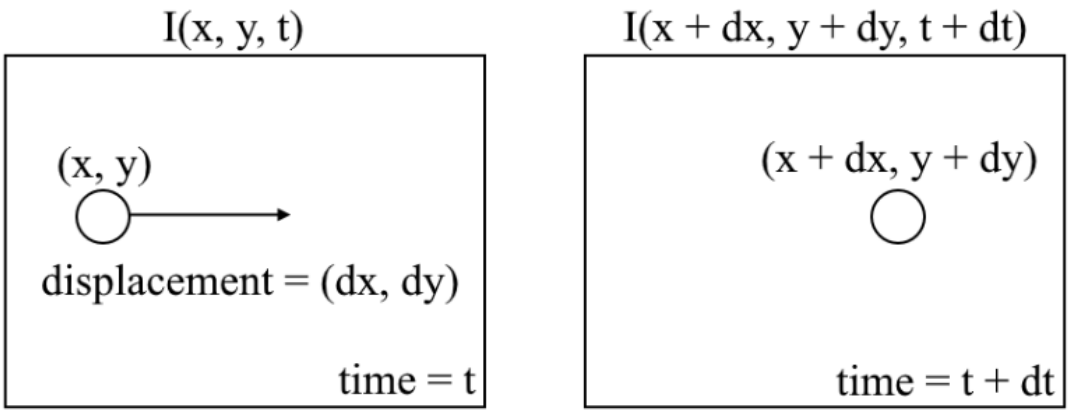


Figure 2.1: Illustrates the difference in displacement between two images mathematically(12)

In consecutive frames, I represents image intensity with respect to space (x,y) and time (t). The first image is I(x,y,t) and its pixels move be (Δx, Δy) over (t) time then the new image is I(x+Δx, y+Δy, t+Δt)(12).

Equation 2.1: The brightness constancy constraint equation represents pixel intensity invariance between two consecutive frames in optical flow analysis.

$$I(x, y, t) = I(x + \Delta x, y + \Delta y, t + \Delta t)$$

Removing common terms by taking the taylor series approximation;

Equation 2.2: Optical flow constraint equation derived via Taylor series approximation, establishing a linear relationship between spatial and temporal image gradients and flow velocities (u, v).

$$\frac{\partial I}{\partial x}u + \frac{\partial I}{\partial y}v + \frac{\partial I}{\partial t} = 0$$

I(x,y,t) image intensity at position (x,y) and time t

$\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y}$ spatial gradients of the image

$\frac{\partial I}{\partial t}$ - temporal gradient

u - flow in the x-direction

v - flow in the y-direction

Different methods to estimate optical flow include: Lucas-Kande method, Horn-Schunck method and Farnebäck(13).

Farnebäck

Swedish mathematician Gunner Farnebäck introduced this method in his 2003 paper(14). His method is classified as a 'dense optical flow method' as it accounts for every pixel within the frame, unlike other algorithms that zone in on a specific feature, like corners, for example.

The algorithm employs a multi-level resolution strategy, constructing a Gaussian pyramid of the input images. Each level of the pyramid stores the image at progressively lower resolutions. This structure allows the algorithm to capture motion across multiple spatial scales. This step helps the algorithm detect all sizes of displacements.

At each level of the image pyramid, the algorithm approximates the neighbourhood of each pixel using a quadratic polynomial. Given two consecutive frames, the algorithm estimates how the polynomial coefficients change due to motion. This modelling assumes that intensity variations can be locally described by a second-degree polynomial, which provides a compact representation of structural changes caused by motion between frames.

Equation 2.3: Quadratic polynomial used in Farnebäck's dense optical flow method to approximate pixel neighbourhoods in motion estimation.

$$f(x) \sim x^T A x + b^T x + c,$$

Where A is a symmetric matrix, b a vector and c a scalar.

An initial estimate of pixel displacement (optical flow vector) is then obtained. The algorithm iteratively refines the estimate using the following steps:

The polynomial model of the first frame is displaced according to the current motion estimate and compared with the polynomial model in the second frame. The difference in intensity values is minimized by updating the displacement vectors.

This process continues until convergence, until the predicted and actual values exhibit minimal discrepancy, thereby yielding a refined motion estimate for each pixel.

Motion vectors computed at each pyramid level are used to refine and propagate estimates from coarse to fine resolutions. This process helps integrate motion information across spatial scales, improving the algorithm's ability to resolve large displacements while maintaining high accuracy at finer levels.

2.2.3 Cross Correlation

Cross-correlation is a mathematical technique used to measure the similarity between two signals as a function of a time-lag applied to one of them(15). It is widely used in signal processing, time-series analysis, and pattern recognition to detect time delays or align similar patterns in data.

Mathematically, cross-correlation is defined as the integral of the product of two signals, where one signal is time-shifted relative to the other:

Equation 2.4: General continuous-time cross-correlation function measuring similarity between two time-shifted signals.

$$R_{12}(\tau) = \int_{-\infty}^{\infty} x_1(t) x_2^*(t-\tau) dt = \int_{-\infty}^{\infty} x_1(t+\tau) x_2^*(t) dt$$

$x_1(t)$ and $x_2(t)$ represent the two signals being compared.

$x_2^*(t)$ denotes the complex conjugate of $x_2(t)$, which is relevant for complex-valued signals.

τ is the delay (or lag) variable.

$R_{12}(\tau)$ quantifies the similarity between $x_1(t)$ and $x_2(t)$ when the second signal is shifted by τ.

The cross-correlation function provides a measure of alignment between the two signals. A high value of $R_{12}(\tau)$ indicates strong similarity at that lag τ.

Equation 2.4 illustrates example cross-correlation measurements. The first column shows the case when the two signals are non-overlapping, resulting in no correlation. The second column shows partial overlap, leading to a small correlation value. Finally, the third column shows perfect alignment (τ=), resulting in maximum correlation.

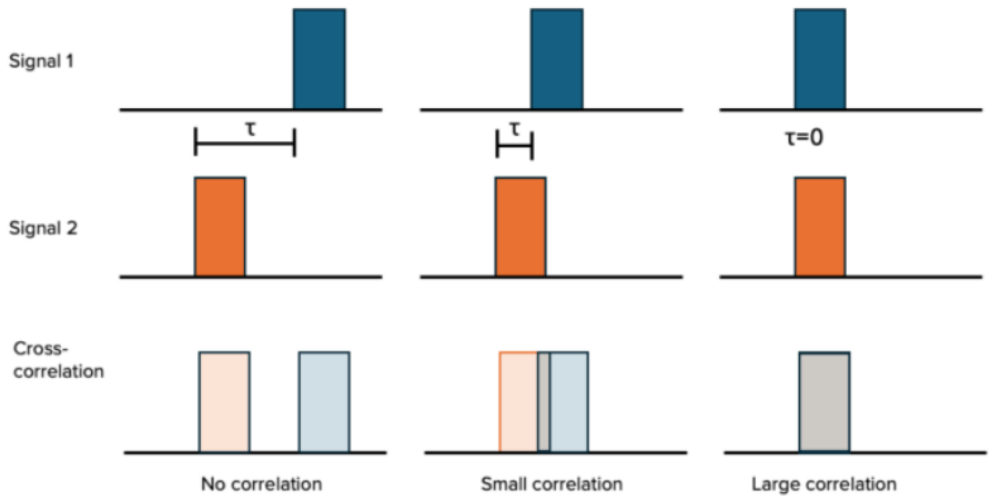


Figure 2.2: Example cross-correlation measurements (15)

2.2.4 Contrast Correlation

Within image processing specifically, template matching and feature similarity, contrast and correlation are often combined through normalised cross-correlation (NCC). Contrast correlation is the comparative analysis of variation in intensity (brightness variation) between images. Contrast correlation is implemented through normalised cross-correlation, which adjusts for both brightness (mean) and contrast (variance) differences

between two signals or image patches(16).

Equation 2.5 Pixel-wise zero-mean and unit-variance normalisation applied to each frame to ensure consistent contrast metrics.

$$R_{norm}(\tau) = \frac{\sum_t (x_1(t) - \bar{x}_1)(x_2(t-\tau) - \bar{x}_2}{\sqrt{\sum_t (x_1(t) - \bar{x}_1)^2 \sum_t (x_2(t-\tau) - \bar{x}_2)^2}}$$

$x_1(t), x_2(t)$: the two signals or image patches being compared.

$\bar{x}_1, \bar{x}_2$ the means of the respective signals (i.e., average brightness).

τ: the lag or shift (in 1D) or spatial offset (in 2D).

The numerator is the centred cross-correlation.

The denominator normalises by the contrast (standard deviation) of both signals.

2.3 Artificial intelligence

Artificial intelligence is currently a speciality in the computer science sector focused on replicating human intelligence. Through experimentation, AI can be tested with problems to optimise for future iterations. Machine learning is a subdivision of artificial intelligence that focuses on algorithms and statistical models allowing computers to learn from and make predictions or decisions based on data without being explicitly programmed. Deep learning is a subset of machine learning that uses neural networks with many layers (deep neural networks) to analyse various levels of data abstractions(17). Examples of deep learning include Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs) and Transformer models. Support vector machine and random forest are under the machine learning umbrella, but specifically, a machine learning algorithm under

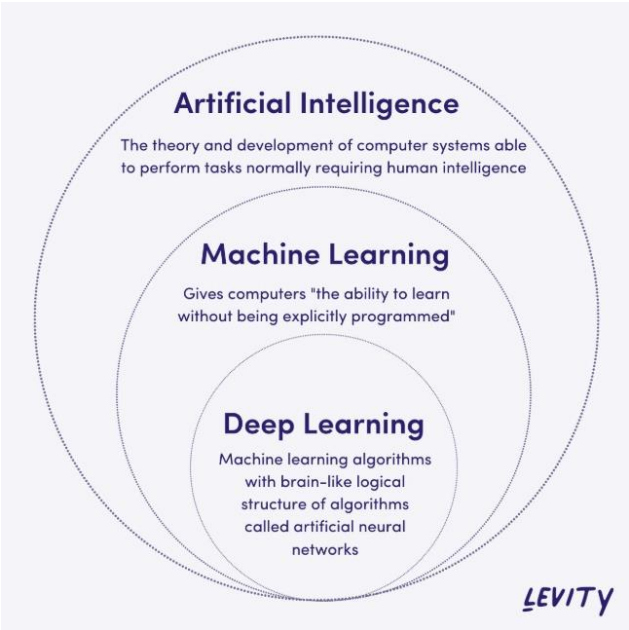
supervision. 

Figure 2.3: Machine Learning is a type of Artificial Intelligence. Deep Learning is an especially complex part of Machine Learning. Source:(18)

What distinguishes deep learning algorithms compared to machine learning algorithms is the lower need for human intervention, and the larger data requirements due to the algorithm structure(18).

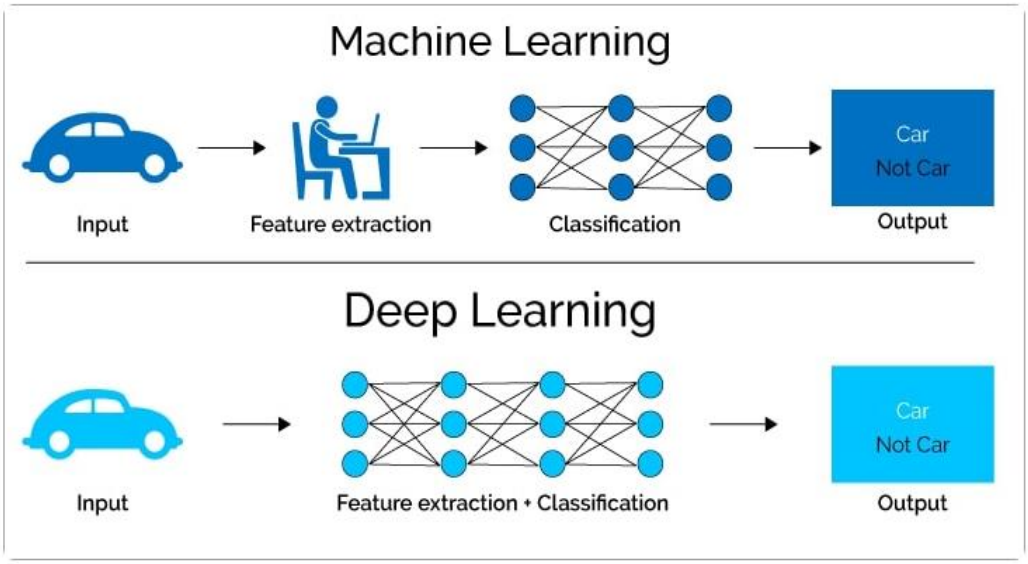


Figure 2.4: Deep Learning algorithms don't need an engineer to identify features, but are

capable of automatic feature engineering through their neural network. Source:(19)

2.3.1 Neural Network

The term neural network was first used in 1943 by mathematician Walter Pitts and neurophysiologist Warren McCulloch in a paper published by them(20). Together they came up with their concept on how neurons work. To show this, the first mathematical model of a neural network was created. The paper describes how a neural network with immense computational power can be created from the use of thresholds and weights applied to simple elements (21). The paper and research in the immediate future after it was published focused more on neuroscience than computer science; however, over the decades, neural networks have been established as a term and staple in computer science. The term 'neural network is universal in deep learning modalities. The term deep learning refers to the depths of layers within a neural network that have only begun. Depths of up to 50 layers were only established in modern times due to the hardware and GPUs increase in power and efficiency compared to 2/3 layers of the 1980s(22).

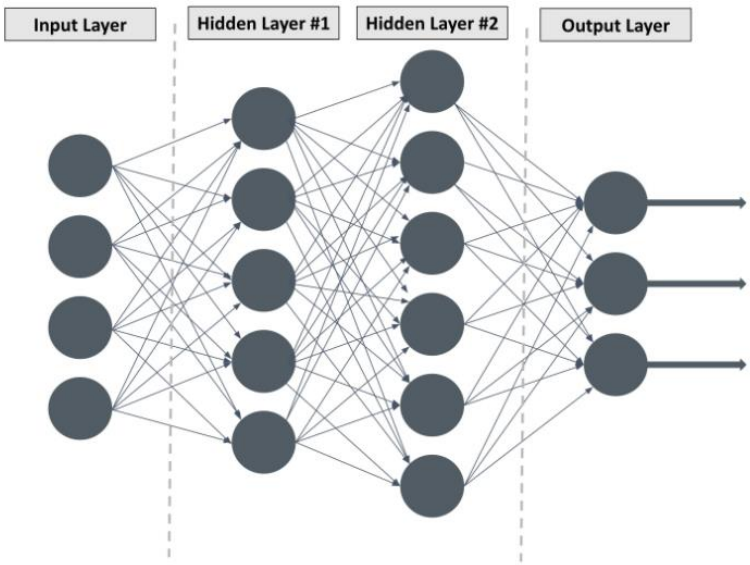


Figure 2.5: A simple artificial neural network

Figure 5 shows a simple neural network with 4 layers. Data manipulation occurs in the

‘hidden layers’. Each layer consists of nodes that make up the neural network. In modern times, networks will consist of at least 10 layers all assorting through different data points. If the output of any individual node is above the specified threshold value, that node is activated, sending data to the next layer of the network. Otherwise, no data is passed along to the next layer of the network.

Each individual node is assigned a specified threshold value. The node is activated if the output is above the specified value of said node. Once activated, data is sent to the next layer of the network. If data reaches a layer and the node is not activated, no data is sent to the next layer (23). This continues until the output layer, then what data is left/created can then be manipulated by the user. Each node is assigned a model, composed of input data, a bias (or threshold), weights, and an output like a linear regression model(23). An example of a formula for a node is below.

Linear Combination (Input to Activation Function):

Equation 2.6: Weighted sum input to a neuron in a neural network, where weights represent feature importance.

$$z = \sum_{i=1}^{n} w_i x_i + b$$

Output Function (Binary Step Activation Function):

Equation 2.7: Simple binary step activation function determining node activation in a feedforward network.

$$f(x) = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{if } z < 0 \end{cases}$$

- x_i - The i-th input feature (e.g., pixel value, word count, etc.)
- w_i - The weight associated with the i-th input (learned during training)
- n - The total number of input features
- b - The bias term (acts like the intercept in a linear equation)
- z - The weighted sum of inputs plus bias (a.k.a. the logit or linear output)
- $f(x)$ - The output of the model (1 or 0, depending on the threshold)

Weights in a neural network determine the importance of a variable(s). The larger the

weight the more it will contribute to the output compared to the other inputs All inputs are then multiplied by their respective weights and then summed(23). The output is then sent through an activation function, which determines the output. If the output is higher than given threshold, it activates the node, passing data to the next layer in the network. The system works by making the output of one node the input of the next node. This process of data being passed from one layer to the next layer defines this neural network as a feedforward network(23).

Variables that change the size of dataset best model include; complexity of the problem, complexity of the learning algorithm, number of classes, number of input features, number of model parameters and dataset compared to model skill(24). Rule of thumb for a sufficient training data set is to use the ten times rule. Input data should be at least 10 times more than the number of degrees of freedom a model has(25). Degrees of freedom is the number of parameters within the dataset. For example if the algorithm in question has distinguishes images of arms from images of legs based on 500 parameters, following the rule, at least 5,000 images would be needed to train the model(25).

2.3.2 Supervised vs unsupervised training

Traditional machine learning algorithms are mathematical models that learn patterns from data. Most effective when problem is simple and well defined. There are four types of traditional machine learning algorithms that fall out of the deep learning bubble, Supervised Learning, Unsupervised Learning, Reinforcement, Learning and Semi Supervised Learning (26).

Supervised learning is a type of algorithm that uses labelled data. Input and desired output are given to the algorithm in to predict and calculate accuracy of outcomes. Supervised tasks performed are either regression or classification tasks. Classification tasks have an output of discrete values that are categories or labels. Regression tasks uses an algorithm to predict a continuous value as an output based on the input variables(27).

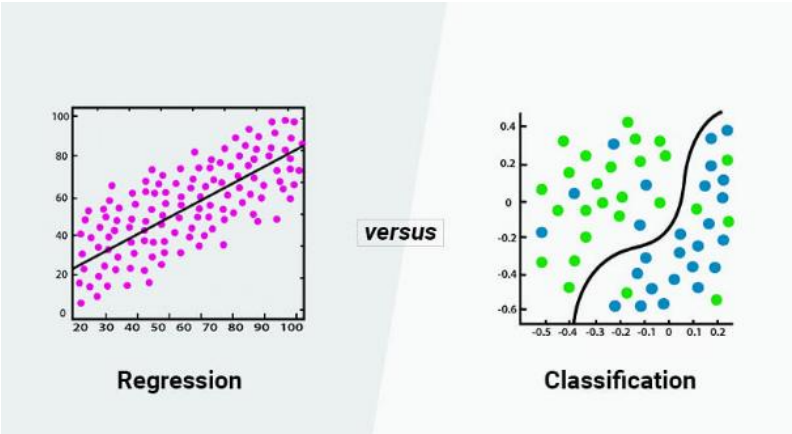


Figure 2.6: Regression task vs classification task source(28)

Examples of supervised learning are support vector machine and logistic regression for classification tasks, linear regression for regression tasks and random forest and decision trees use both classification and regression tasks(26).

Unsupervised learning is a type of algorithm that finds patterns and hidden structures amongst unlabelled data to group them in a sensible manner. Unsupervised learning does not use trained data sets but instead uses direct techniques to predict outcomes(19). Unsupervised learning is used in clustering tasks. K-means clustering is an example of a clustering algorithm.

Semi-supervised learning algorithms use both labelled and unlabelled data. Semi uses a relatively low amount of labelled data with a high amount of unlabelled data for training. The algorithm uses labelled data for the context of the task. Unlabelled data then generalises the algorithm with a constant flow of unseen examples. Algorithms in reinforcement learning go through trial and error to optimise themselves to gain the best possible outcome.

2.3.2 Support Vector Machine

Support Vector Machine (SVM) is a supervised learning algorithm, where both the input data X and corresponding labels y are known during training. Support vector machine works primarily with two defined classes in a binary fashion. SVM works by assigning data on a plane based on the commonality of features between all the data points. A support vector machine then classifies data into predicted classes by finding the best

'hyperplane' to separate data into classes. The hyperplane is drawn as a margin in the dimension separating data points. Support vectors are data points closest to the hyperplane; these are the data points most difficult to classify(29). Support vectors influence hyperplane placement the most.

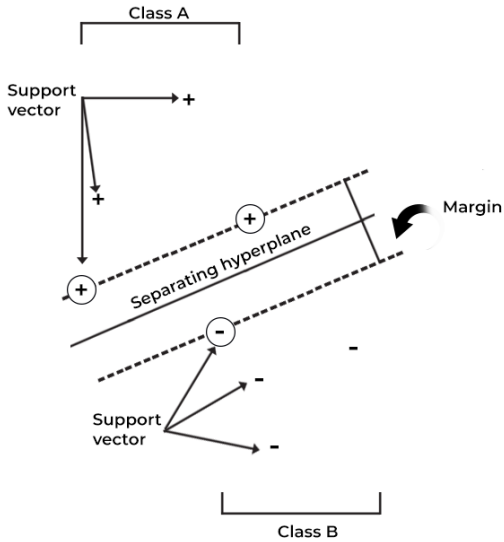


Figure 2.7: SVM plane. source(30)

The margin seen above is the separating distance parallel to the hyperplane. The distance measured is between the hyperplane and the support vector on both class sides. The wider the margin, the better the generalisation, because the model is less likely to overfit. In fact, maximising the margin is the core objective in SVM training. A large margin reduces the model's sensitivity to noise in training data.

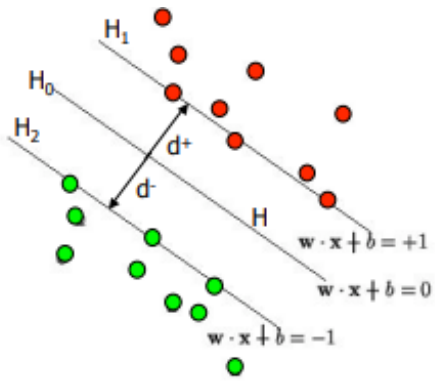


Figure 2.8: A Hyperplane within a Support Vector Machine boundary (29)

A hyperplane in SVM is the decision boundary that separates two classes. In its general form:

Equation 2.8: General form of a hyperplane in an n-dimensional feature space, where

$$w \cdot x + b = 0$$

- $w$is the weight vector
- x is the input vector
- b is the bias (offset)

To separate data points belonging to different classes, SVM defines two marginal hyperplanes:

Equation 2.9: Marginal constraints defining the region separating class +1 and class -1.

- For class $+1$:

$$w \cdot x_i + b \geq +1 \qquad \text{when } y_i = +1$$

- For class -1

$$w \cdot x_i + b \geq -1 \qquad \text{when } y_i = -1$$

Hyperplanes are defined that:

- $H_1: w \cdot x + b = +1$
- $H_2: w \cdot x + b = -1$
- Decision Boundary (H_0 or H):

$$w \cdot x + b = 0$$

$-$This lies halfway between H_1 and H_2

The hyperplane is the median of H_1 and H_2. Moving a support vector moves the decision boundary. Other vectors' movement has no effect on the decision boundary

or the hyperplane formed.

For any point x, the perpendicular distance to the hyperplane is:

Equation 2.10: Perpendicular distance from a point to the hyperplane in Euclidean space.

$$\text{Distance} = \frac{|w \cdot x + b|}{\|w\|}$$

- Distance between H_0 and H_1 (or H_2):

Equation 2.11: Margin width between the two supporting hyperplanes. Maximising this margin improves model generalisation.

$$\frac{2}{\|w\|}$$

The goal is to maximise margin to minimise classification error; thus, the need is to minimise $\|w\|$ under the condition that there are no datapoints between H_1 and H_2 (soft margin):

To maximise the margin (distance between H_1 and H_2), we need to minimise $\|w\|$ under the constraint that:

Equation 2.12: Hard-margin SVM constraint ensuring all data points lie outside or on the margin.

$$y_i(w \cdot x_i + b) \geq 1 \quad \text{for all } i$$

This is the hard-margin SVM. If we allow some misclassifications (soft margin), we introduce slack variables.

In real-world data, perfect separation may not be possible. Soft-margin SVM introduces slack variables and a regularisation parameter C to allow some misclassifications, improving generalisation on noisy datasets.

Data point mapping relies on kernels. When the data is not linearly separable in the original feature space, kernels allow SVM to operate in a transformed feature space where a linear decision boundary can be found, without explicitly computing the transformation. Shown above is linear SVM, but other kernels that can be used include polynomial kernel,

RBF kernel and sigmoid kernel. Kernel choice significantly impacts performance (31).

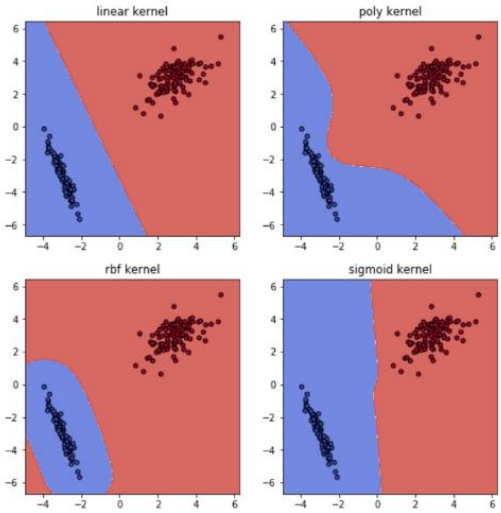


Figure 2.9: Kernel types in SVM. source(32)

2.3.3 Random Forest

Random forest is an example of an ensemble method. In general, ensemble methods work by combining the predictions of multiple base estimators built with a given learning algorithm to improve the generalisability over a singular estimator(33). The random forest algorithm works by combining the output of multiple decision trees to reach a single result. Random forest has a reduced risk of overfitting when there is a high number of decision trees. This works because the averaging of uncorrelated trees will reduce the overall variance and prediction error(34).

Assumptions made in RF:

Decision trees are independent of each other (achieved through bootstrap sampling and feature randomness)

There is sufficient data for RF to build a diverse tree and achieve optimal performance

Trees grow deep enough to resolve underlying patterns within data

Noise is not systemic in trees but is randomly distributed(35).

As random forest can do both classification and regression tasks to a high level of

accuracy, it is a preferred method of machine learning among data scientists who need flexibility. RF eases analysis of random forest effectiveness by making the variable importance, or contribution, to the model transparent. Gini importance and mean decrease in impurity (MDI) can be manipulated to see model accuracy when variables are added or removed.

In regression tasks, RF performs a mean square error for each node to decide which branch is best for the decision.

Equation 2.13 Mean Squared Error used by Random Forest in regression tasks to optimise node splitting.

$$\text{MSE} = \frac{1}{N}\sum_{I=1}^{N}(fi - yi)^2$$

- Where N is the number of data points,
- fi is the value returned by the model
- yi is the actual value for data point i

Classification tasks use the Gini index.

Equation 2.14 Gini Impurity Index used for evaluating the homogeneity of splits in classification trees.

$$\text{Gini}=1 - \sum_{i=1}^{C}(p_i)^2$$

Probability and class are used to calculate the Gini of each branch of a node, determining which outcome is more likely to occur.

2.4 Confusion Matrices

Within supervised machine learning, particularly for classification tasks, evaluating model performance is critical for both academic rigour and real-world deployment. The confusion matrix serves as an informative diagnostic framework that can highlight performance metrics beyond simple accuracy, including precision, recall (sensitivity), specificity, negative predictive value (NPV), and prevalence.

A confusion matrix is a table that visualises a classification model's predictions against

the actual labels. For binary classification, this matrix consists of four fundamental elements(36):

True Positives (TP): Instances where the model correctly predicts the positive class.

True Negatives (TN): Instances where the model correctly predicts the negative class.

False Positives (FP): Instances where the model incorrectly predicts the positive class.

False Negatives (FN): Instances where the model incorrectly predicts the negative class.

Table 2.2: Labelling of a Confusion Matrix

		Predicted Value		
		0/No	1/Yes	
Actual Value	0/No	True Negative	False Negative	Recall
	1/Yes	False Positive	True Positive	Specificity
	Prevalence	Precision	Negative Predictive Value (NPV)	Accuracy

Accuracy

Accuracy is the most used performance metric, calculated as:

Equation 2.15: Accuracy formula calculating the proportion of total correct classifications in a binary classification task.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

While important, accuracy can be deceptive, especially when there is class imbalance. A model might predict the majority class most of the time and still achieve high accuracy, despite failing to detect the minority class entirely.

Precision

Precision measures the reliability of positive predictions:

Equation 2.16: Precision metric representing the proportion of predicted positives that are actually correct.

$$\text{Precision} = \frac{\text{TP}}{\text{TP+ FP}}$$

This metric is relevant in domains where false positives are costly, such as fraud detection or medical diagnostics, where overdiagnosis can have psychological or economic consequences.

Recall (Sensitivity)

Recall quantifies the ability of a model to correctly identify actual positives:

Equation 2.17: Recall (Sensitivity) metric indicating the model's ability to correctly identify positive cases.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

In applications where missing a positive case is unacceptable, like cancer detection, high recall is crucial. Models optimised purely for recall often sacrifice precision, leading to more false positives.

Specificity

Specificity is the counterpart to recall, measuring the ability to correctly identify negatives:

Equation 2.18: Specificity metric quantifying the proportion of true negatives among all actual negative cases.

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

This metric is key when the negative class carries equal or greater importance, such as confirming the absence of a disease or condition with high certainty.

Negative Predictive Value (NPV)

NPV assesses how often a predicted negative result is actually correct:

Equation 2.19 : Negative Predictive Value measuring the reliability of a negative prediction.

$$\text{Negative Predictive Value (NPV)} = \frac{\text{TN}}{\text{TN} + \text{FN}}$$

This mirrors precision and is useful in evaluating the reliability of a model's negative predictions in sensitive contexts like medical screening.

Prevalence

Prevalence refers to the proportion of actual positives in the dataset:

Equation 2.20: Prevalence metric expressing the proportion of actual positive cases in the dataset.

$$\text{Prevalence} = \frac{\text{TP} + \text{FN}}{\text{Total Samples}}$$

Prevalence helps in calibrating model thresholds and avoiding misleading performance interpretations, especially in imbalanced datasets.

Practical Considerations

In practice, the choice of which metric to prioritise depends on the application context. For example:

- High recall is vital in medical applications where missing a diagnosis could be fatal.
- High precision is favoured in legal or financial contexts where a false alarm may be costly or damaging.
- Balanced accuracy and F1-score (harmonic mean of precision and recall) are often used when neither metric can be prioritised exclusively.

2.5 Key Related Studies in Non-Invasive Brain Activity Monitoring

Paper (37) depicts an experiment closely linked to one conducted in this paper, with cerebral haemodynamic change being recorded via temporal spatial analysis.

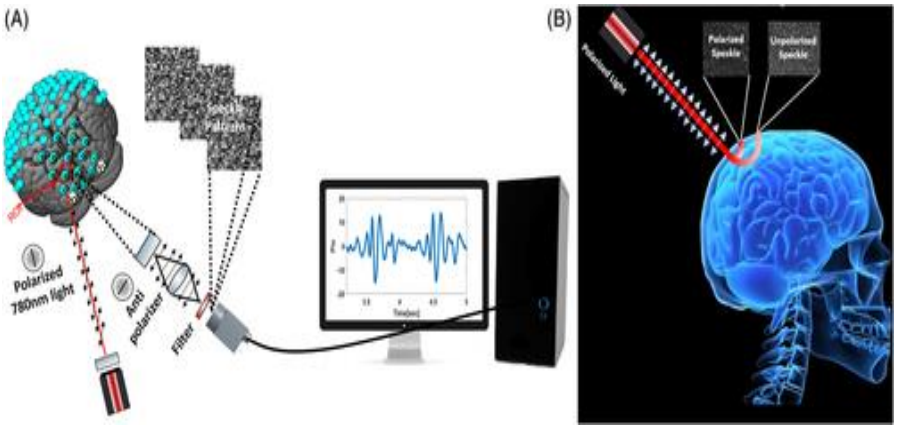


Figure 2.10: Illustrates experimental set-up with functional near-infrared spectroscopy where a laser beam illuminates the scalp whilst the subject is made to perform tasks that will induce haemodynamic responses in the visual cortex (occipital lobe).

When a laser illuminates the brain, a camera is in place to record back-scattered light (i.e., speckles) generated at the brain's blood vessels. Conclusions from the study show a precedent for remote and non-contact methods for monitoring brain activity by detecting nano-vibrations associated with haemodynamic changes. The key reason why this paper shows an avenue for evolution is that the system showed high sensitivity to small-scale mechanical movements in brain vessels, removing any uncertainty that responses to tasks were by coincidence.

Paper (38) shows a similar study. The study used 8 healthy participants between 29 and 74 years old and recorded their brain activity under the conditions: rest without sensory stimulation, and under stimulation of each sense (smell, taste, and hearing). To stimulate the senses, the subjects were exposed to smell: alcohol-soaked cotton ball, taste: Sweet chocolate, and hearing: continuous constant-frequency noise. Each test session lasted 10 seconds, and data was recorded five times per sense for each subject, resulting in approximately 240,000 frames. The area of interest captured by the camera Basler saa1300-200um was the cerebral cortex, with a laser source of green light 532nm 770 µW illuminating the skull. As a control an EEG headset (OpenBCI EEG headband with Ganglion bio-sensing 4-channel board) was worn but subjects to test whether a deep neural network classifier could extract the same results with reasonable accuracy. Image below depicts the experiment.

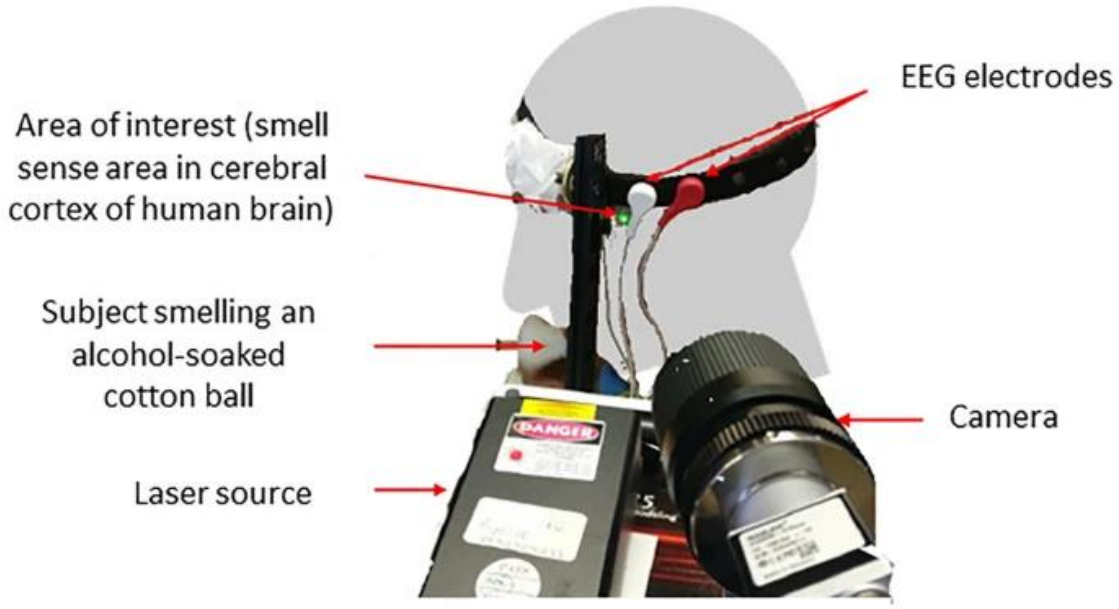


Figure 2.11: Experiment set up from the paper (38).

Table 2.3: Details the results of models from the paper (38).

Model	Classes	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	Inference time (ms)
Single speckle frame convnet	Sense	82	85	78	82	4
	No Sense		79	83		4
Full video speckle convnet	Sense	89	93	91	89	830
	No Sense		84	87		830
This model	**Sense**	**95**	**98**	**92**	**95**	**2**
	No Sense		**92**	**98**		**2**

Table 2.4: Details the results of the EEG model from the paper(38).

EEG Model	Precision (%)	Recall (%)	F1-Score (%)	Accuracy (%)
Sense	0.75	1.00	0.86	83
No-Sense	1.00	0.67	0.80	83

Compared to other methods, this DNN outperformed single-frame CNN and full-video CNN in accuracy and speed of detection. The model was also compared with a CNN-based EEG model trained on the simultaneously recorded EEG data, with the results showing high conformity between the EEG and speckle-based models. Overall, the study offers a promising approach for the non-invasive and remote detection of sensory-related brain activity, providing a potential tool for medical diagnostics, cognitive research, and brain-computer interface development.

Neurophotonics is the merging of optics and neuroscience, with its applications including invasive and non-invasive techniques. Paper (39) Focus on non-invasive neurophotonic methods that use near-infrared light to probe functional activity in the brain, specifically, functional near-infrared spectroscopy, fast optical signal and diffuse correlation spectroscopy. The different optical techniques have different optical parameters to detect change, and as these parameters differ, the change in brain activity needed to record a response is different. Fast optical signal works by recording the scattering between the membrane potential and Ion and water flux, which performs action potential crossing the membrane. Diffuse correlation spectroscopy works by detecting the doppler shift of blood velocities within cerebral blood flow. Functional near-infrared spectroscopy works by detecting the absorption of cerebral blood oxygenation, with the physiological parameters being total haemoglobin, oxygenated haemoglobin and deoxygenated haemoglobin (40).

'Analysis of Neural Interactions Explains the Activation of Occipital Cortex by an Auditory Stimulus' by A. R McIntosh, published in 1988, describes the activation of occipital cortex (part of the occipital lobe) by an auditory stimulus (41). This paper illustrates that when human subjects are under auditory stimulus, a visual event is signalled that activates the left dorsal occipital cortex. When the cortex is activated,

regional cerebral blood flow increases. This paper is relevant as it gives precedent to the fact that with stimulus created and detected as such by the human brain, it will be converted into impulse signals that increase cerebral blood flow that can be used as a measurement for brain state.

'Dual-Wavelength Confocal Laser Speckle Contrast Imaging Using a Deep Learning Approach' by Du, E. ; Zheng, Haohan ; He, Honghui search by orcid ; Li, Shiguo ; Qiu, Cong ; Zhang, Weifeng ; Wang, Guoqing search by orcid ; Li, Xingquan ; Ma, Lan ; Shen, Shuhao ; Zhou, Yuan details the development of a system with the ability to measure blood flow and blood oxygen saturation with their method, BlingNet. BlingNet is a blood flow imaging CNN based on the laser speckle contrast imaging technique and deep learning approach, which, according to results, was able to achieve superior accuracy and robustness with higher imaging speed compared to the traditional nonlinear fitting method, making real-time blood flow imaging possible (42). The imaging setup consisted of two light sources in a dual-wavelength confocal laser speckle contrast imaging system. 'Two solid-state single-frequency lasers with a wavelength of 640 nm (DL640-100-S0, CrystaLaser, Reno, NV, USA) and 785 nm (DL785-300-S0, CrystaLaser, Reno, NV, USA), respectively.' Then, a dichroic mirror is placed to merge the two laser beams into the same optical path. The beam is then expanded to cover a larger illumination area. For their deep learning framework, they collected 93 sets of samples containing data from 31 different flow rates. Due to the high quality of each frame, 2560 × 2160, more memory and resources were required to which led to memory overflow during training. To circumvent this, they performed the window sliding technique.

'Ergodic speckle contrast optical coherence tomography velocimetry of rapid blood flow' by Jiachi Hong, Wenting Zhu, Kaikai He, Xiao Chen, Jinling Lu, and Pengcheng Li details ergodic speckle contrast optical coherence tomography (ESCOCT) to visualize 3D blood flow velocity fields, demonstrating the utility of motion-based analysis in haemodynamic studies (43).

'Low Frequency Systemic Haemodynamic "Noise" in Resting State BOLD fMRI: Characteristics, Causes, Implications, Mitigation Strategies, and Applications' by Yunjie Tong Lia M Hocke Blaise B Frederick presents use of cross correlation for examining haemodynamic changes (44). The paper discusses how systemic haemodynamic

fluctuations affect the accuracy and reliability of cross-correlation analyses used to evaluate functional connectivity. Within the discussion, the importance of addressing these physiological artefacts is highlighted as they can introduce spurious correlations between brain regions, compromising the validity of fMRI-derived functional networks. Different methods for noise mitigation are explored, including regression-based approaches, independent component analysis (ICA), and physiological modelling. In laser speckle imaging (LSI), similar challenges arise due to motion artefacts

, variations in blood flow, and environmental interference, which can affect the precision of speckle-based cross-correlation measurements (44). By adopting advanced noise correction techniques discussed in this paper, the reliability of cross-correlation for assessing focus and rest states based on cerebral blood flow dynamics can be improved. This aligns with the broader goal of improving the accuracy and clarity of results in non-invasive optical imaging applications.

In 'Laser Speckle Contrast Imaging of Cerebral Blood Flow, ' Andrew K. Dunn presents how LSCI performance is influenced by factors such as ambient lighting, optical scattering, and variations in tissue optical properties (1). Dunn emphasises the sensitivity of contrast correlation metrics to external lighting conditions, which can introduce variability in speckle contrast measurements and, subsequently, affect the accuracy of blood flow quantification. Moreover, the paper highlights the need for rigorous calibration and controlled imaging environments to mitigate these influences and ensure data reliability.

Rehabilitation devices for quadriplegia often use EEG as the input function. Quadriplegics have little to no control of voluntary movements below the neck, to help fill out commands they cannot physically do, specialised assistive devices are built. With the use of brain computing interfaces, devices can be built by assessing neurological activity to determine what the user needs at that instant. Paper (45) details experimentation of detection of cognitive states as a command for a wheelchair. The paper states they want to develop a 'BCI device that adaptively adjusts to the psychological effects of the patient'. They had an EEG attached to the frontal, central, temporal, parietal, and occipital portions of the brain/skull. Stimuli of relaxing noise, loud noise and no noise were used to induce cognitive states of relaxation, stress and neutral, respectively.

*Figure 2.12: (45) Mental state images captured during stimulus provided to the user to segregate EEG data for three different psychological states: (**a**) normal state, (**b**) relaxed state, and (**c**) stressed state. Images are captured using the Emotiv BrainViz software version 1.1.*

Motor imagery Based Functions	Wheelchair commands
Push	Forward
Rotate left	Left
Rotate right	Right
Neutral	Stop
Mental state (excited)	Speed up
Mental state (relaxed)	Speed down

Figure 2.13: (45) Command mapping for EEG BCI-controlled wheelchair

2.5.1 Summary of Key Related Studies in Non-Invasive Brain Activity Monitoring

The relevant findings of each paper have been summarised in three sections: methodology/apparatus, data processing method and experimental findings. These can be found in Table 2.5:

Table 2.5: Summary of Key Related Studies in Non-Invasive Brain Activity Monitoring

Ref	Apparatus	Data processing method	Experimental findings
(37)	Continuous-wave NIR laser (fNIRS band) illuminating occipital scalp; high-speed speckle camera placed off-axis for non-contact capture of back-scattered "nano-vibration" patterns during visual tasks	Spatio-temporal speckle-pattern analysis; cross-correlation of minute axial displacements	Demonstrated that tiny haemodynamically driven surface oscillations track visual-cortex activation, establishing a contact-free, high-sensitivity alternative to conventional fNIRS-based haemodynamic monitoring

(38)	532 nm, 770 µW laser illuminating cerebral cortex; Basler saA1300-200 µm camera (240 kfps dataset); OpenBCI 4-channel EEG headband used in parallel; olfactory, gustatory & auditory stimuli (10 s blocks × 5)	Deep neural network (DNN) on speckle video; compared with single-frame CNN, full-video CNN, and CNN trained on synchronous EEG	DNN on speckle video hit 95 % accuracy (2 ms inference) vs. 89 % and 82 % for the two CNN baselines; results paralleled the EEG classifier, validating remote optical sensing of stimulus-linked brain activity
(39)	Survey of non-invasive neuro-photonic systems: fNIRS, fast optical signal (FOS), and diffuse correlation spectroscopy (DCS); typical CW/FD lasers in 650-900 nm range	Parameter-specific signal models (absorption, scattering, Doppler shift) with modality-tailored inverse algorithms	Clarifies how each optical parameter dictates the minimum detectable functional change; highlights complementary depth/speed trade-offs across fNIRS, FOS and DCS modalities
(41)	^15O-water PET (regional cerebral blood-flow tracer) while subjects received isolated auditory tones	Structural equation modelling of inter-regional neural interactions	Auditory stimulus selectively raised rCBF in left dorsal occipital cortex, providing an early physiological precedent for cross-modal (audio-to-visual) activation that later optical methods can exploit
(42)	Dual-wavelength confocal LSCI: 640 nm & 785 nm single-frequency lasers merged by dichroic mirror, beam-expander, sCMOS (2560 × 2160) detector	BlingNet CNN for simultaneous blood-flow & SaO_2 estimation (window-sliding to fit GPU memory)	Real-time imaging with higher accuracy/robustness than nonlinear fitting; enables single-shot flow + oxygenation mapping—key for bedside monitoring

(43)	Ergodic speckle-contrast OCT system (broadband source, axial interferometer) for volumetric micro-vascular imaging	Ergodic speckle-contrast velocimetry extracting full 3-D flow vector fields	Resolved rapid flow (> 10 mm s^{-1}) in 3-D with OCT-native depth resolution; expands speckle velocimetry beyond surface-integrated maps
(44)	Resting-state BOLD fMRI (3 T) with concurrent PPG & respiration traces	Cross-correlation, regression of physiological regressors, independent component analysis (ICA)	Systemic haemodynamic oscillations (< 0.15 Hz) inflate functional-connectivity metrics; paper catalogues mitigation strategies directly portable to speckle-correlation studies of focus vs. rest
(45)	14-channel EEG cap covering frontal, central, temporal, parietal, occipital lobes; audio stimuli (relaxing sound, loud noise, silence)	Deep neural network classifier of spectral-temporal EEG features	Accurately distinguished neutral, relaxed, stressed states for hands-free wheelchair control; underscores value of multichannel neural input when optical access is impractical

CHAPTER 3 METHODOLOGY

3.1 Participants

This study uses seven healthy adult volunteers. Volunteers are exclusively males with short hair, 20-35. This experiment was conducted in accordance with the University of Glasgow's Project Safety.

3.2 Data Acquisition

The dataset was obtained using a high-speed Basler acA640-750uim camera and a single-channel OBIS 808 SWIR laser diode operating at 50mW power. The laser was directed at the occipital lobe of participants while they underwent a visual stimulus protocol consisting of alternating 15-second intervals of resting (dark grey screen with a crosshair) and cognitive stimulation (circular black-and-white pattern). Each trial lasted 90 seconds, comprising 45 seconds of visual stimulus and 45 seconds of rest, repeated three times for different laser positions. Laser speckle frames of size 128×128 pixels were acquired at 1550 frames per second, resulting in 139,500 frames per data acquisition, saved in HDF5 format for further analysis.

3.3 Data processing

In this study, two pipelines were employed for brain state classifications. Two custom-made, one for traditional machine learning algorithms and the other for a convolutional neural network (CNN). The output of the two lines differs in dimensionality and representation of the input data.

In both pipelines, raw laser speckle image sequences were first pre-processed to extract temporal signals. Each imaging sequence was recorded at 1550 frames per second and processed into six 15-second intervals per 90-second trial, alternating between relaxing and cognitive stimulus conditions. Each trial consisted of alternating between 2 cognitive states every 15-second interval: relaxing and paying attention, with a total of 6 segmentations per file. To preserve this structure, each temporal signal was segmented into 6 intervals per file, with segments labelled accordingly.

For a 90-second trial (1 HDF5 file), Labelling follows:

Table 3.1: Labelling for a singular trial/HDF5 file

Interval	Time Range (sec)	Label	Label Index
1	0-15	Relaxing	1
2	15-30	PayingAtteniton	1
3	30-45	Relaxing	2
4	45-60	PayingAttention	2
5	60-75	Relaxing	3
6	75-90	PayingAttention	3

For each interval, a continuous scalar signal was derived using one of three inter-frame analysis techniques: contrast correlation, cross-correlation, or optical flow (discussed in the following sections). These scalar signals formed the basis for further feature extraction and classification. In total, classifiers have 126 segments (21 trials with 6 segments each) to use as both training and testing. Segments were split into either 70/30 or 80/20 dependent on the classifier used.

In this study, three distinct signal extraction methods were implemented to process laser speckle imaging (LSI) data prior to classification: Optical Flow, Cross-Correlation, and Contrast Correlation.

3.3.1 Optical Flow

In this study, two distinct applications of optical flow are used. The first, which is detailed in this section, involves deriving 1D motion trajectories from dense optical flow as a time-series signal suitable for traditional machine learning. The second approach, discussed later, focuses on optical flow-based image generation used as input for a convolutional neural network (CNN). While both use the Farnebäck method, they serve different analytical purposes and employ different data representations.

This section describes the 1D trajectory-based implementation of optical flow using the Farnebäck dense optical flow algorithm, which is well-suited to tracking subtle pixel-level motion across time in laser speckle recordings.

MATLAB's built-in *'opticalFlowFarneback'* function was used, which estimates motion between two consecutive grayscale frames. Each frame is rescaled to the range [0,1] to standardise pixel intensity values:

Equation 3.1: Normalises pixel intensity values to the range [0,1] across all frames to ensure numerical stability during flow estimation.

$$I_{scaled} = \frac{I - \min(I)}{\max(I) - \min(I)}$$

This normalisation keeps numerical stability across frames with varying brightness(16). Once scaled, each frame is passed to *'estimateFlow'*, which computes dense motion vectors for every pixel based on Farnebäck's polynomial approximation. With this, a local neighbourhood of pixel intensities is approximated by a quadratic polynomial:

Equation 3.2: Represents local pixel neighbourhoods using a second-order polynomial

$$I(x) = x^T A x + b^T x + c$$

$x \in \mathbb{R}^2$ is the pixel position

A $\in \mathbb{R}^{2X2}$ is a symmetric matrix representing second-order spatial variation,

$b \in \mathbb{R}^2$ captures the gradient,

$c \in \mathbb{R}$ is a constant offset.

Between two successive frames, the change in the polynomial coefficients indicates motion. The displacement d is solved from:

Equation 3.3: Describes how the change in gradient vectors between frames relates to pixel displacement using the polynomial model.

$$b_2 = b_1 - 2A_1 d \implies d = \frac{1}{2} A_1^{-1}(b_1 - b_2)$$

This gives a dense motion vector field where each pixel has a flow vector (u,v), representing estimated velocity in the x and y directions(46).

In this implementation, magnitude and orientation is extracted from the flow field for each pixel and convert them to a complex number representation:

Equation 3.4: Converts flow magnitude and direction into a complex number for each pixel to facilitate summation.

$$\text{vector}(i) = \text{Magnitude}(i) \cdot e^{j \cdot \text{Orientation}(i)}$$

Summing this across all pixels yields a global displacement estimate per frame:

Equation 3.5: Sums complex flow vectors over the image to estimate global motion in each frame.

$$\text{Vect}_{\text{global}} = \sum_{x,y} (\text{flowMagnitude}_{x,y} \cdot e^{j \cdot \text{flowOrientation}_{x,y}})$$

The real and imaginary parts are accumulated over time to produce two continuous signals:

Equation 3.6: Accumulates real and imaginary components over time to generate 1D motion trajectories.

$$x(i+1) = x(i) + \Re\left(\text{Vect}_{global}(i)\right), \quad y(i+1) = y(i) + \Im(\text{Vect}_{\text{global}}(i))$$

Together, x(t) and y(t) form the 1D motion trajectories, capturing temporal changes in motion strength and direction. The trajectories convey the dynamic evolution of speckle motion over time and are thought to correspond to local blood flow changes or particle movement(47).

To isolate physiologically relevant components, both x(t) and y(t) were passed through a 10th-order Butterworth bandpass filter with a low-stop frequency of 0.02 Hz and a high-stop frequency of 3 Hz, designed in MATLAB.:

```
fdesign.bandpass('N,F3dB1,F3dB2', 10, low_stop, high_stop, f_sample);
```

This removed high-frequency noise and slow baseline drifts, improving the interpretability of the temporal signal.

Finally, a single unified signal is derived from the filtered components:

Equation 3.7: Calculates the magnitude of motion at each time point, forming a final scalar signal input for machine learning.

$$\text{Motion(t)} = \sqrt{x(t)^2 + y(t)^2}$$

This 1D signal is then segmented into 15-second intervals based on the experimental protocol, with alternating periods of *relaxing* and *paying attention*. These segments are used for statistical, frequency-based, and wavelet features extraction for machine learning algorithms.

3.3.2 Cross-correlation

2D cross-correlation was used to extract 1D temporal signal from laser speckle image sequences.

Two consecutive frames, $f_{t-1}(x, y)$ and $f_t(x, y)$, were zero-mean normalised:

Equation 3.8:Zero-mean normalisation of each frame by subtracting its average pixel intensity.

$$\tilde{f}_t(x, y) = f_t(x, y) - \mu_t \text{a}$$

where μ is the mean pixel intensity of frame t.

Equation 3.9:Computes the similarity between two consecutive normalised frames using 2D cross-correlation.

$$R_t = xcorr2(\tilde{f}_{t-1}, \tilde{f}_t)$$

This operation slides one frame over the other in both spatial dimensions and computes similarity at each shift location. From the resulting 2D correlation matrix R_t only the maximum value was retained:

This operation involves sliding one frame over another in both spatial dimensions and calculating similarity at each shift position. From the 2D correlation matrix R_t, the max value recorded was retained:

Equation 3.10: Extracts the peak value from the correlation matrix to represent temporal

similarity between frames.

$$c(t) = \max(R_t)$$

The scalar value $c(t)$ quantifies how well the two frames align in spatial structure. High values indicate minimal change, meaning both signals have a high correlation, while lower values suggest motion, pattern shifts, or temporal noise. Repeating this across the full recording produces a 1D correlation time series.

To isolate physiologically relevant components, both x(t) and y(t) were passed through a 10th-order Butterworth bandpass filter with a low-stop frequency of 0.02 Hz and a high-stop frequency of 3 Hz, designed in MATLAB.:

```
fdesign.bandpass('N,F3dB1,F3dB2', 10, low_stop, high_stop, f_sample);
```

The filtered signal was then segmented into six 15-second intervals for each file, matching the experimental structure of alternating relaxing and paying attention states.

3.3.3 Contrast correlation

Inter-frame contrast correlation was used to extract a 1D temporal signal from laser speckle image sequences. This method compares the contrast patterns of consecutive frames to detect changes in speckle structure over time, which is often indicative of motion or flow (48).

To standardise intensity, pixel values were zero-mean and unit-variance normalised for each pair of consecutive frames, $f_{t-1}(x, y)$ and $f_t(x, y)$.

Equation 3.11:Standardises each frame by adjusting pixel values to have zero mean and unit variance.

$$\tilde{f}_t(x, y) = \frac{f_t(x, y) - \mu_t}{\sigma_t}$$

μ_t is the mean intensity of frame t

σ_t is the standard deviation of pixel intensities in frame t

This normalisation ensures that each frame's contrast attributes are consistent before comparison. The similarity between consecutive normalised frames was then calculated

using an element-wise dot product.

Equation 3.12:Computes the similarity of contrast patterns between consecutive frames using dot product correlation.

$$c(t) = \frac{1}{N}\sum_{i=1}^{N} \tilde{f}_{t-1}(i) \cdot \tilde{f}_t(i)$$

N represents the total number of pixels in each frame (flattened into a 1D vector). The resulting scalar value $c(t) \in [-1,1]$ represents the contrast correlation coefficient, with values close to 1 indicating high similarity and lower values suggesting structural motion or temporal decorrelation.

Repeating this process across all frames created a continuous 1D contrast correlation signal over time.

To isolate physiologically relevant components, both x(t) and y(t) were passed through a 10th-order Butterworth bandpass filter with a low-stop frequency of 0.02 Hz and a high-stop frequency of 3 Hz, designed in MATLAB.:

```
fdesign.bandpass('N,F3dB1,F3dB2', 10, low_stop, high_stop, f_sample);
```

The filtered signal was then segmented into six 15-second intervals for each file, matching the experimental structure of alternating relaxing and paying attention states.

3.4 Data processing for Traditional Machine Learning Algorithms vs Convolutional Neural Network

Traditional machine learning algorithms retained 1D time-series format from data processing techniques above. These time series then went through a forward sequential feature selection algorithm to extract the most influential features of the optical flow data. Those features were then fed to the classifiers to train and test the machine learning models. Models were trained on a 70/30 split of segments.

Deep learning-based classification was also explored as an alternative to traditional machine learning algorithms, with the aim of learning abstract and nonlinear spatial representations of cerebral blood flow signal plots. The CNN pipeline requires data in the form of images as input. 1D temporal signals were transformed into 2D visual format.

Each signal segment that was processed with either optical flow, cross correlation or contrast correlation went through an additional step of being converted into a plot image and other media, capturing the full temporal dynamics of the scalar metric. These images were resized to a uniform input dimension of 224×224 pixels to be compatible with the architectures of pre-trained CNN models. Each plot was labelled according to its condition (relaxing or stimulus) and stored in a structured directory for CNN model. Models were trained on an 80/20 split due to CNN being a layered network, requiring as much data as possible for generalising.

3.5 Traditional Machine Learning Algorithm

To classify between the two cognitive states of relaxing and under visual stimulus, two traditional machine learning algorithms were

tested on engineered features extracted from pre-processed cerebral blood flow signals. Regardless of the data processing technique applied, the downstream classification pipeline followed a consistent feature engineering methodology.

Feature Extraction

All segments were rigorously analysed for binary classification. A total of 23 features were selected from the following categories:

Statistical descriptors: mean, standard deviation, variance, mean squared value, skewness, kurtosis, third moment

Frequency-domain features: power spectrum, magnitude of FFT, dominant frequency, mean frequency, average phase, total spectral power, and the mean signal value (as a basic FFT coefficient)

Wavelet-based metrics: mean, standard deviation, variance, and entropy of wavelet coefficients (db1)

Signal shape descriptors: gradient energy, mean autocorrelation coefficient, maximum FFT peak amplitude, and minimum FFT trough

Percentile features: 25th and 75th percentiles

Dimensionality Reduction and Feature Selection

To avoid overfitting and improve classifier performance, feature selection was applied in two stages: Principal Component Analysis and Relief-F. Principal Component Analysis (PCA) was used to reduce redundancy and retain 95% of the variance in the dataset. Relief-F algorithm ranked features based on their relevance to class distinction. The top 23 features were retained for model input.

This hybrid approach ensured that both variance-based dimensionality and class relevance were optimised for each data processing technique configuration.

3.5.1 Support Vector Machine (SVM)

SVM models were implemented with MATLAB's fitcsvm() function. Each model was trained using a 70/30 (126 samples total, 89 for training, 37 for testing) holdout split, and standardisation was applied to all features.

Initially, all SVM models were tested using RBF kernel. However, when tested with features from Optical Flow and Cross Correlation, RBF kernel performance was poor, with accuracy values below 50 percent, which was determined to be the threshold between blind guessing and calculated contextual guessing by the algorithm(49). RBF was attempted initially on all three data processing techniques as it was hypothesised that the higher-dimensional feature landscape generated by these techniques lacked the separability or curvature required for the radial basis transformation.

As a result, a pivot was made to using a linear kernel for SVMs trained on Optical Flow and Cross Correlation features. This change significantly improved performance, as the linear kernel better matched the structure of the extracted features for these methods. SVM trained on Contrast Correlation performed exceptionally well with RBF kernel, resulting in no pivot for this configuration. This was likely due to the highly separable and low-noise output of the contrast-based signals, which aligned well with the nonlinear boundary created by the RBF kernel.

All models used the following optimisation settings:

- KernelFunction: 'linear' or 'rbf' depending on data type
- KernelScale: 'auto'
- Standardise: true
- ClassNames: [0, 1]

3.5.2 Random Forest (RF)

Random Forest classifiers were trained using MATLAB's TreeBagger() function with the following parameters:

- Number of Trees: 500
- OOBPrediction: Enabled
- MinLeafSize: 5 (to reduce overfitting and improve generalisation)
- NumPredictorsToSample: All

As with SVM, the RF classifier was trained and validated using a 70/30 (126 samples total, 89 for training, 37 for testing) holdout split. The same set of 23 Relief-F-selected features were used for training. Predictions on the test set were converted to numeric values for comparison and evaluation against ground-truth labels.

3.6 Convolution Neural Network (CNN)

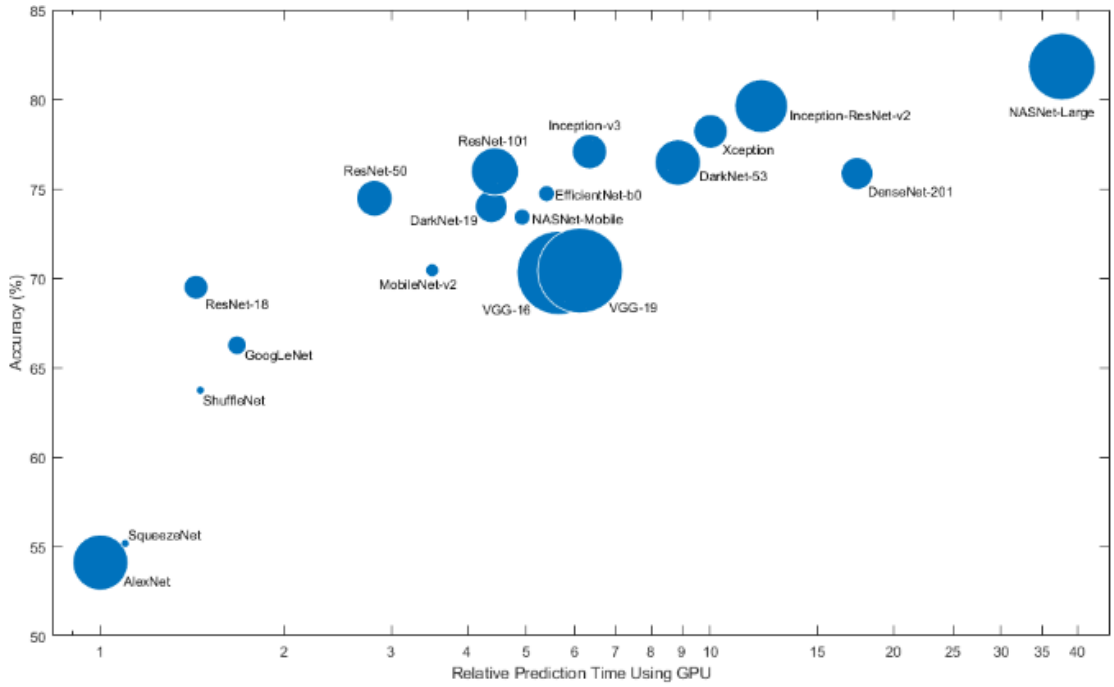


Figure 3.1: CNN options in MATLAB(50)

Neural networks are used for either classification, feature extraction or transfer learning. Figure 3.1 shows a comparison of pretrained neural networks relative to one another with the parameters of accuracy and relative prediction time using GPU. VGG-16 and ResNet-18 was chosen for classification tasks as they're reported to have the high accuracy whilst also having few layers. This balance of accuracy and layers was crucial due to the low amount of data samples that is available for the network. Larger neural networks like ResNet-50 have approximately 25 million parameters which for a small data set of images that lack visual complexity is prone to overfitting(51). Overfitting is where during training when there is lack of details for neural network to assign to a class it will begin to memorise noise, minor variations, or sample-specific features in the training set rather than learning general patterns that generalise to new, unseen data(52). ResNet-18 has 11 million parameters making it a more viable option for this project(53). To capture the difference between the two to see if more parameters would equal to an increase in accuracy, VGG-19 was used alongside ResNet-18. VGG-19 has approximately over 138 million parameters(54).

3.6.1 VGG19

Key implementation details:

- Network structure: Final three layers replaced with custom classification head
- Transfer learning: Fully connected and SoftMax layers retrained for binary classification
- Optimizer: Adam
- Mini-batch size: 8
- Max Epochs: 30
- Learning Rate: 1e-4
- L2 Regularization: 1e-4 (to penalise large weight updates)

3.6.2 ResNet-18

Key implementation details:

- Network structure: Base model frozen except for final classification layers
- Transfer learning: Fully connected, SoftMax, and classification output layers were replaced
- Optimizer: Adam
- Mini-batch size: 16
- Max Epochs: 10
- Learning Rate: 1e-4

CHAPTER 4 PRESENTING FINDINGS AND RESULTS

4.1 Processing techniques

4.1.1 Optical Flow Farneback

Optical Flow was used to detect changes in cerebral blood flow in the occipital lobe during visual stimulus. CBF increased during the visual stimulus and subsequently decreased during resting periods. These changes can be observed in the images in figure 4.1 and 4.2, where the optical flow of both the x-position and y-position of the laser speckle image pixels are displayed

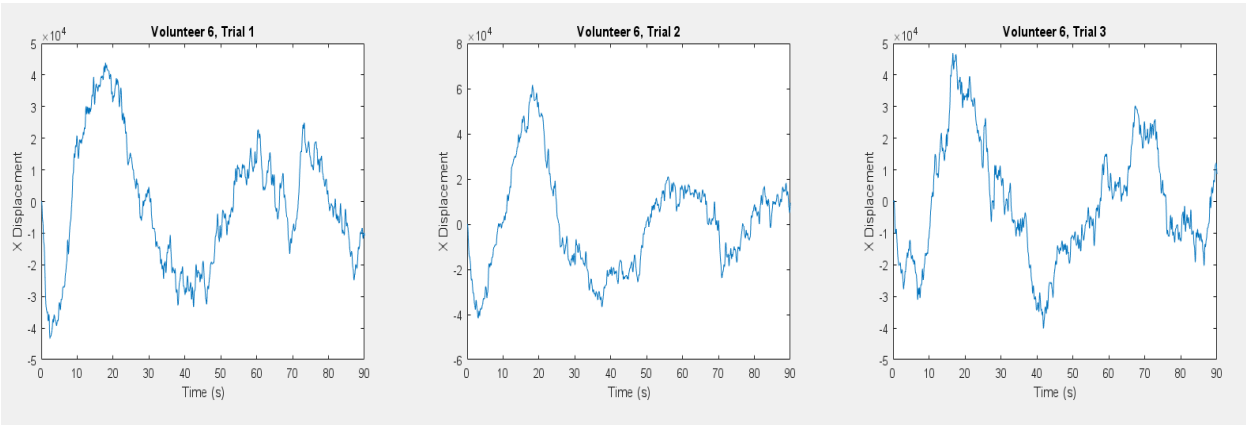


Figure 4.1: Optical flow of the sixth volunteer's laser speckle image pixels in x-position showing amplitude of flow against data acquisition time (seconds).

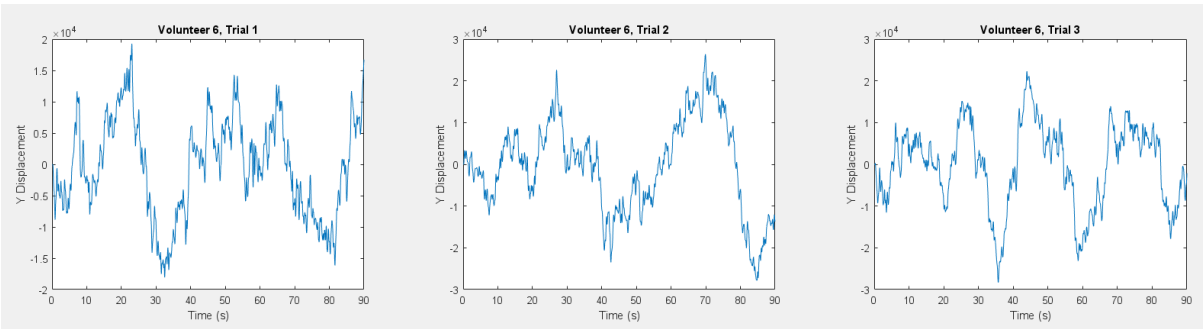


Figure 4.2: Optical flow of the sixth volunteer's laser speckle image pixels in y-position showing amplitude of flow against data acquisition time (seconds).

Figures 4.1 and 4.2 show a considerable increase in CBF inside the occipital lobe

compared to the resting condition. The images show substantial variations in blood flow in response to a visual stimulus, indicating an increase in visual processing in the occipital brain(55). Laser speckle picture pixels' x and y locations vary significantly over time. During the resting state, the x and y positions fall significantly with a negative amplitude. Some anomalies appear in trials 2 and 3, where substantial oscillations do not match either condition in the relevant time frames. These abnormalities can be linked to motion artefacts, which occur when a volunteer moves slightly and skews data results. Another form of error to consider is the external optical feedback effect, which causes noise in laser speckle pictures and reduces optical flow accuracy. Finally, optical scattering and absorption might be sources of inaccuracy because scattering distorts laser speckle patterns, affecting data accuracy. Furthermore, tissue density might cause different measurements to be made depending on where the laser is directed, thus skewing the generated laser speckle pictures.

QUIVER PLOTS

By assigning a motion vector (u, v) to each pixel in the image, the Farnebäck approach generates a dense optical flow field. The u notation represents displacement in the x-direction, while v represents displacement in the y-direction. These vectors illustrate the movement velocity between consecutive frames. The direction of the vector indicates the direction of the motion, and its magnitude reflects the speed. Figure 4.3 illustrates frame by frame optical flow use.

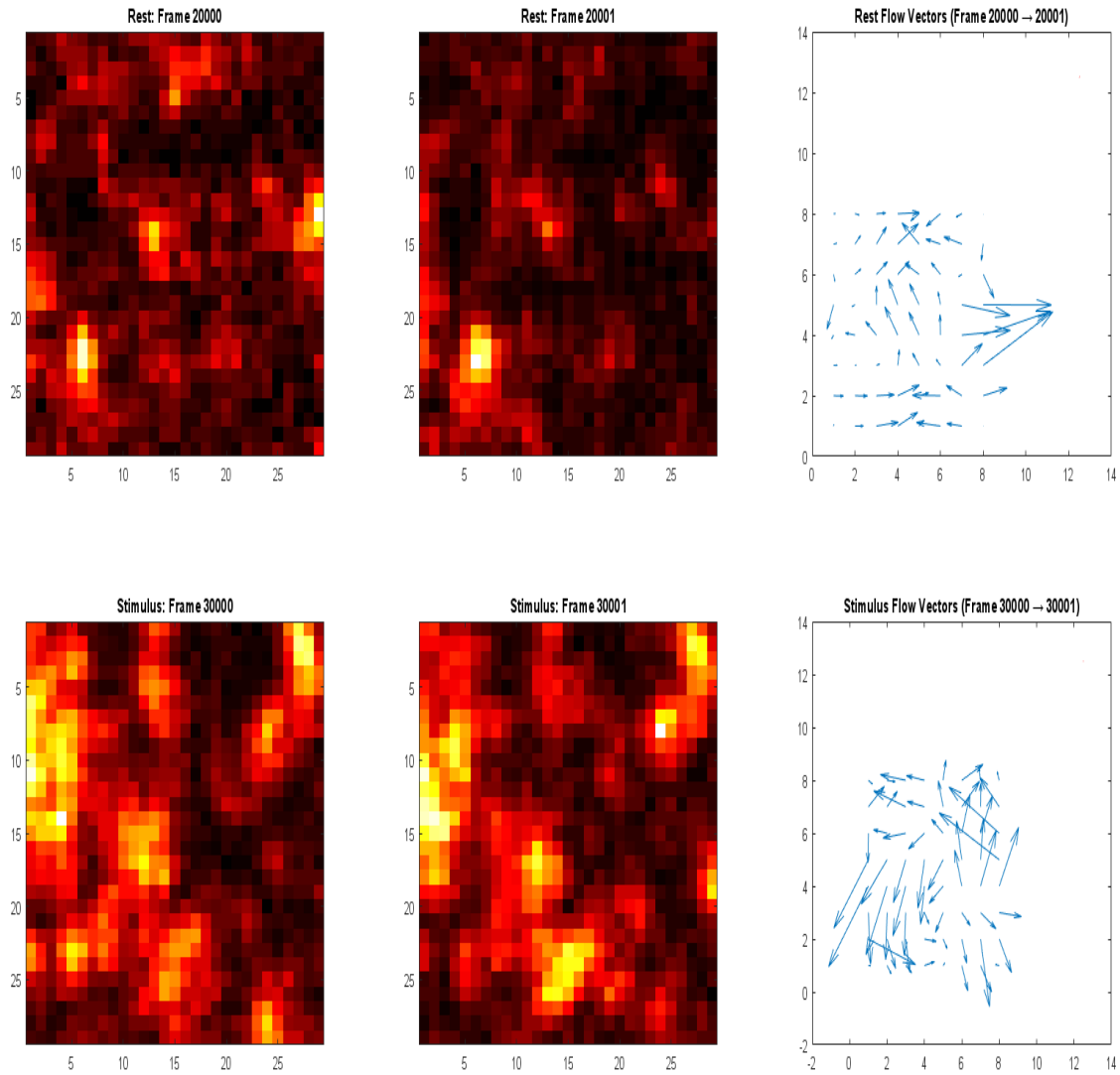


Figure 4.3 Optical flow quiver plots displaying pixel brightness for successive frames during resting period on the top row and visual stimulus bottom row with vector map displaying pixel movement between frames. Obtained from Volunteer 3's 3rd trial.

Figure 4.3 shows the difference in flow vectors when individual is at rest compared to under stimulus. The left and right panels show the blood present in a specified frame and the consecutive frames, respectively, with bright yellow indicating blood flow is present, whereas the dark colours show no movement. The top row is the individual at rest, and the bottom row is the individual under visual stimulus. As expected, there is a higher concentration of CBF (yellow) during visual stimulus than there is in the resting state. Under visual stimulus, blood flow is seemingly highly concentrated but in small pockets, indicating different areas are receiving an intake of blood flow. It is important to note that the bright yellow indicating blood in the rest row is not an abnormality but aligns with

the hypothesis. There is always going to be blood in the occipital lobe as visual processing occurs all the time. It is important to examine the increase in flow as that will indicate the individual is reacting to more visual stimuli

The right panels in figure 4.3 shows the difference between these frames measured by flow vectors. During the resting state, the vectors have much smaller magnitudes when comparing to visual stimulus where the average vector has a larger magnitude indicating much faster CBF. Vectors are larger and move in a relatively unison movement along showing an increase in blood flow movement that would be expected from an individual who is attempting/gaining focus. This adds credence the hypothesis that cerebral blood flow increases under visual stimulus due to increase in neural activity and decreases at resting state. Figures like this can help give better visual understanding as to how CBF moves in the occipital lobe, and how it is affected by neural activity.

4.1.2 CNN input example images

Figure 4.4-4.7 are all 1D signals that have been processed via Optical Flow, Cross Correlation and Contrast Correlation. The figures are from Volunteer 3 Trial 1 with the left side of each figure showing the first 15 seconds of the trial acting as the resting segment and the right side is the 15 seconds thereafter where the volunteer is under visual stimulus.

Optical Flow

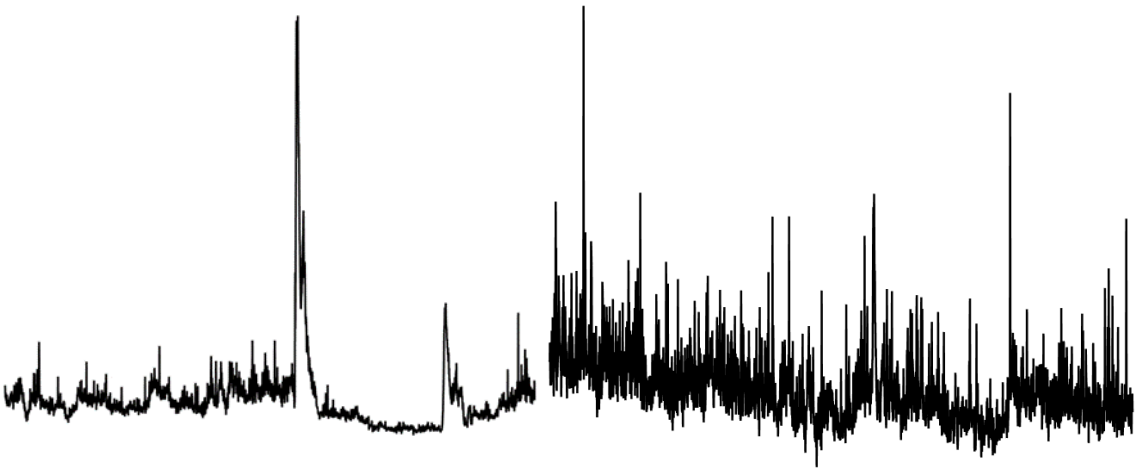


Figure: 4.4 Both images use optical flow processing with output as line plots. Both images are from Volunteer 3's 1st Trial. Left is a 15 second rest segment. Right is the 15 second segment under visual stimulus directly after the rest period.

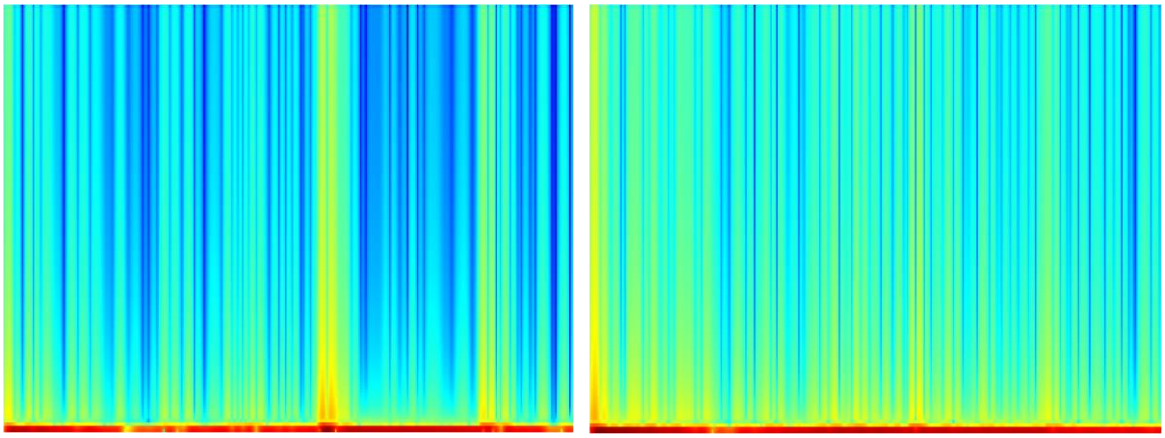


Figure: 4.5 Both images use optical flow processing with output in spectrogram format. Both images are from Volunteer 3's 1st Trial. Left is a 15 second rest segment. Right is the 15 second segment under visual stimulus directly after the rest period.

Cross Correlation

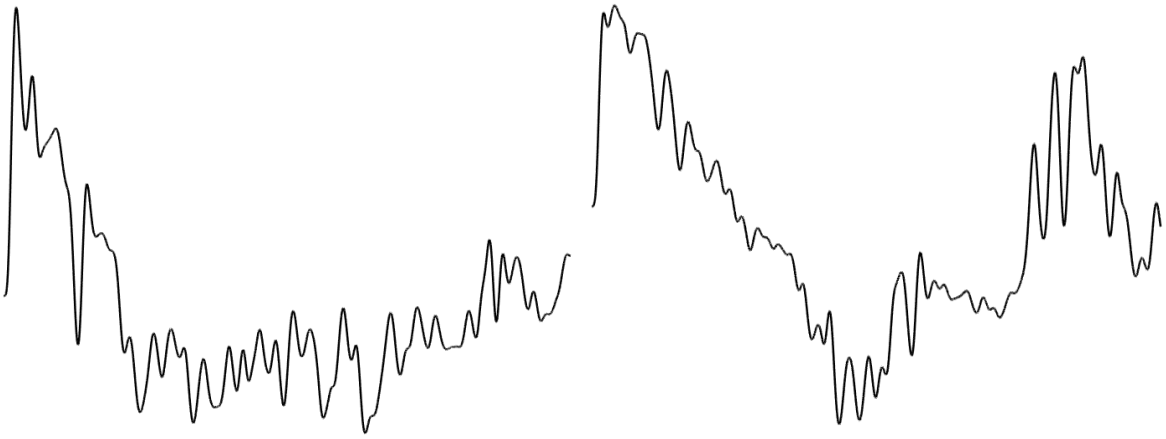


Figure: 4.6 Both images use Cross Correlation with output in line plot format. Both images are from Volunteer 3's 1st Trial. Left is a 15 second rest segment. Right is the 15 second segment under visual stimulus directly after the rest period.

Contrast Correlation

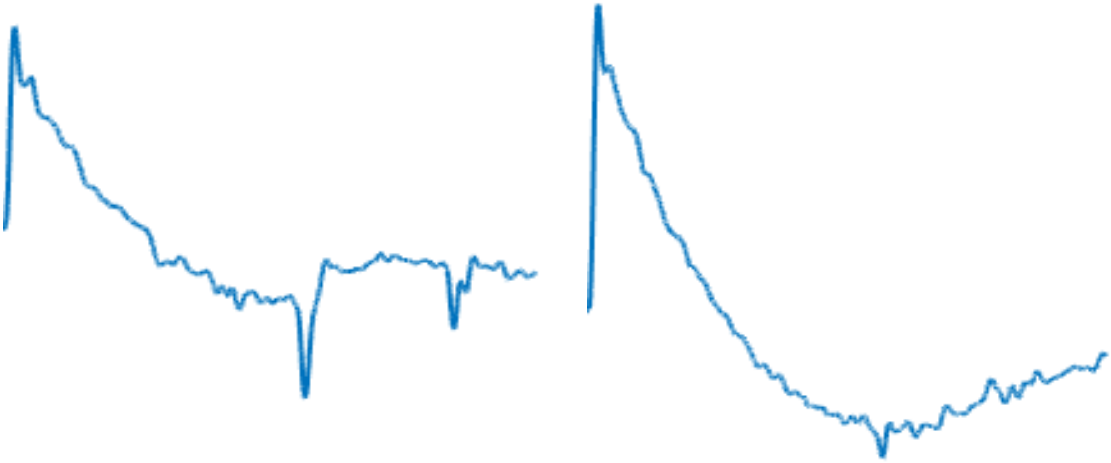


Figure: 4.7 Both images use Contrast Correlation with output in line plot format. Both images are from Volunteer 3's 1st Trial. Left is a 15 second relaxing segment. Right is 15 second segment under stimulus

4.2 Classifier Results

After processing the neuroimaging data using three distinct methodologies (Optical Flow, Cross Correlation, and Contrast Correlation) the next logical step in the investigation was to examine how each signal representation affected the ability of machine learning classifiers to distinguish between resting and stimulated brain states. This should in future, allow data similar in terms of its features to be analysed to determine which images correspond with what brain state, with no human input, and improve the validity of using

machine learning algorithms. Four different classifiers were tested to verify which classifier was best compatible with what data processing technique. The confusion matrices (figures 4.8-4.21) revealed distinct patterns in classifier compatibility and preprocessing efficacy.

4.2.1 How to read the Confusion Matrices

There are 126 segments in total used for classification:

- 63 segments correspond to when the volunteer is visually stimulated.
- 63 segments correspond to when the volunteer is at rest.

These segments were processed using the techniques discussed in the methodology section and classified using a variety of traditional and deep learning classifiers, with exact splits detailed in the methodology. Each configuration outputs a confusion matrix, which allows interpretation of how accurately the classifier distinguishes between the two brain states.

Below is a reference layout for understanding how to read these confusion matrices:

Table 4.1: Labelling of a Confusion Matrix within the context of this study

		Predicted Value	
		Classifiers Predicting Rest segment (0)	Classifier Predicting Visually Stimulated Segment
Actual Value	Segment is a Rest segment (0)	Classifier correctly predicts segment as Rest **(True Negative)**	Classifier incorrectly predicts segment as Stimulated when it is a Rest Segment **(False Negative)**
	Segment is a Visually Stimulated (1)	Classifier incorrectly predicts segment as Rest when it is a Stimulated Segment **(False Positive)**	Classifier correctly predicts segment as stimulated **(True Positive)**

- **True Positive (TP):** Stimulated state correctly identified
- **True Negative (TN):** Resting state correctly identified
- **False Positive (FP):** Rest segment misclassified as stimulus
- **False Negative (FN):** Stimulated segment misclassified as rest

These categories directly relate to classifier sensitivity (recall), specificity, precision, and overall accuracy. The layout for interpreting these matrices follows standard structure (see reference diagram, table 2.2 in Background Section). Across all classifiers, performance was assessed using normalised percentages to enable direct comparison.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{Total Samples}} \times 100$$

- Diagonal cells (TN and TP) indicate correct classifications.
- Off-diagonal cells (FP and FN) indicate misclassifications.

All confusion matrices are presented as percentages, normalised over the total number of test samples in each configuration.

4.2.2 Support Vector Machine

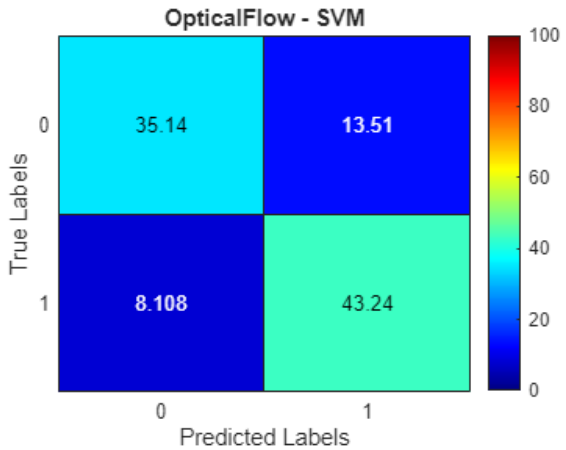


Figure 4.8: Confusion matrix for Optical Flow processing and Support Vector Machine classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy: 78.38%

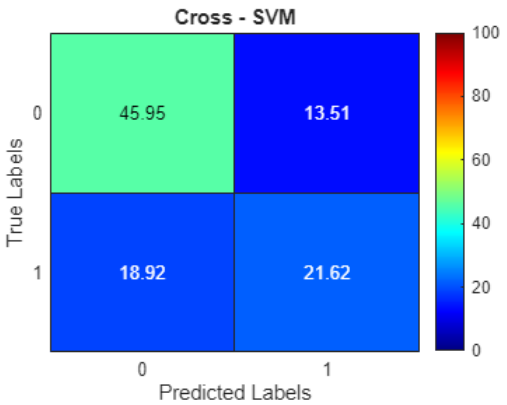


Figure 4.9: Confusion matrix for Cross Correlation processing and Support Vector Machine classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy 67.57%.

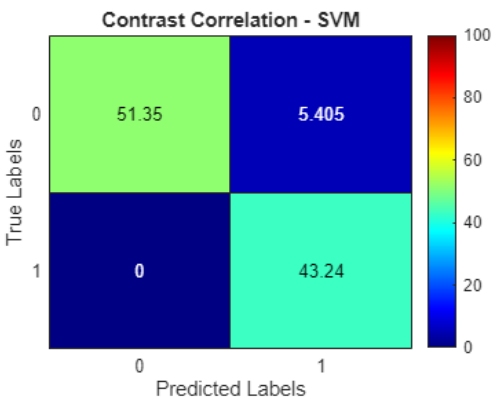


Figure 4.10: Confusion matrix for Contrast Correlation processing and Support Vector Machine classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy: 94.60%.

4.2.3 Random Forest

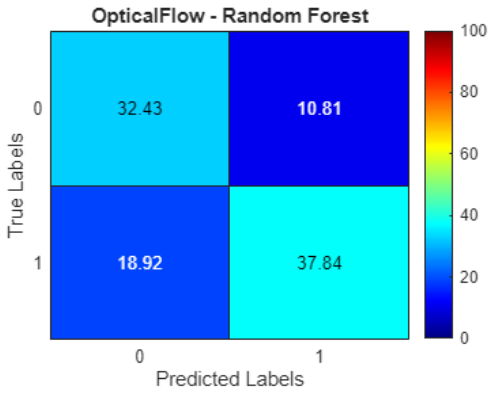


Figure 4.11: Confusion matrix for Optical Flow processing and Random Forest classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy 70.27%

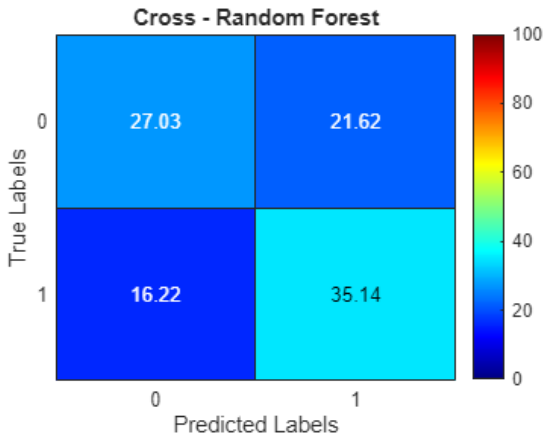


Figure 4.12: Confusion matrix for Cross Correlation processing and Random Forest classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy 62.16%.

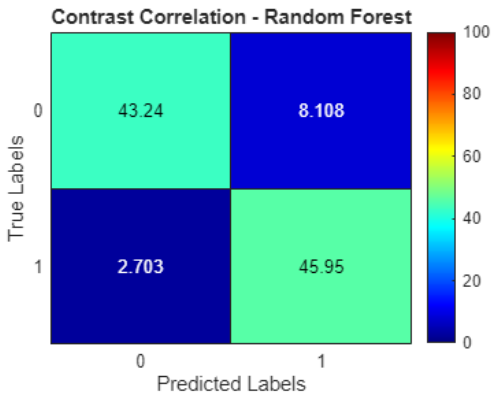


Figure 4.13: Confusion matrix for Contrast Correlation processing and Random Forest

classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy 89.19%.

4.2.4 VGG19

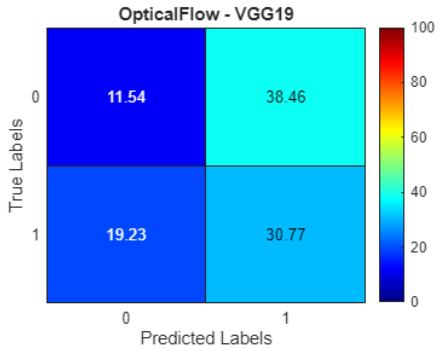


Figure 4.14: Confusion matrix for Optical Flow Line Plot 2D images and VGG19 classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy 42.31%.

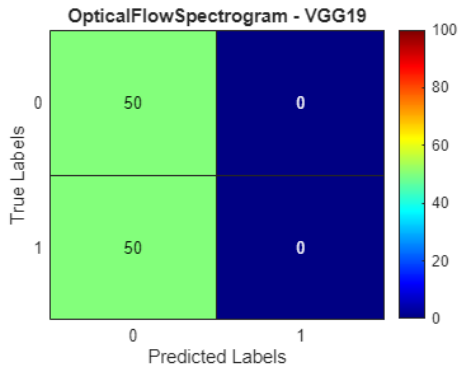


Figure 4.15: Confusion matrix for Optical Flow Spectrogram 2D images and VGG19 classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy 50.00%.

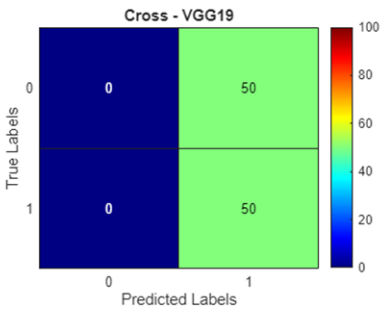


Figure 4.16: Confusion matrix for Cross Correlation Line Plot 2D images and VGG19

classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy 50.00%.

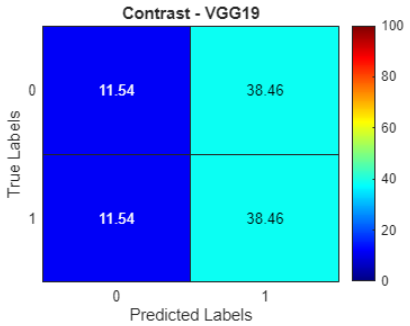


Figure 4.17: Confusion matrix for Contrast Correlation Line Plot 2D images and VGG19 classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy 50.00%.

4.2.5 ResNet-18

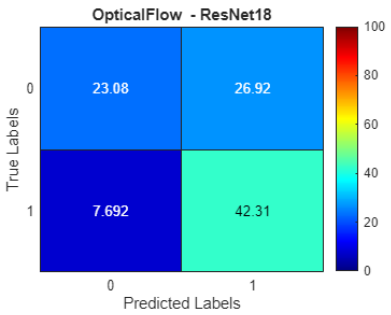


Figure 4.18: Confusion matrix for Optical Flow Line Plot 2D images and ResNet18 classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy 65.38%

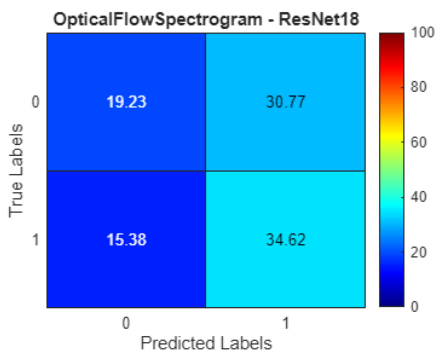


Figure 4.19: Confusion matrix for Optical Flow Line Plot 2D images and ResNet18

classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy 53.85%

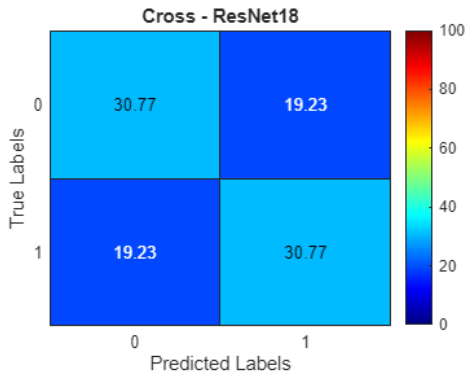


Figure 4.20: Confusion matrix for Cross Correlation Line Plot 2D images and ResNet18 classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy 61.54%

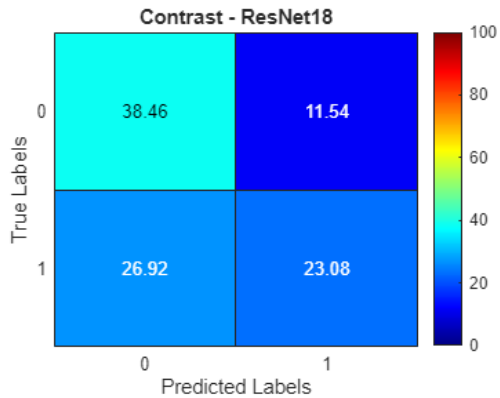


Figure 4.21: Confusion matrix for Contrast Correlation Line Plot 2D images and ResNet18 classifier displaying percentage of correctly predicted labels compared to the true value labels. Test Accuracy 61.54%

4.3 Summary of Classifier Performances with Preprocessing Techniques

Classifier	Preprocessing Technique	Test Accuracy (%)	Comment
SVM (Linear Kernel)	Optical Flow	78.38	Strong linear separability; low false positive rate and balanced

			precision–recall trade-off.
SVM (Linear Kernel)	Cross Correlation	67.57	Moderate accuracy; features lack consistent boundary alignment, resulting in decreased specificity and increased FN.
Support Vector Machine (RBF Kernel)	**Contrast Correlation**	**94.60**	**Best-performing model overall; high true positive rate and excellent class separability in nonlinear space.**
Random Forrest	Optical Flow	70.27	Reliable performance; slight positive bias toward visually stimulated class observed via higher FP rate.
Random Forrest	Cross Correlation	62.16	Weakest among RFs; high variance in features introduces noise, reducing recall and classification robustness.
Random Forrest	**Contrast Correlation**	**89.19**	**Very strong performance; low false negative rate and high sensitivity to contrast-derived features.**
VGG19	Optical Flow 2D Images	42.31	Severe underfitting due to dataset size; validation accuracy stagnates, leading to ineffective generalisation
VGG19	Optical Flow Spectrogram 2D	50.00	Severe underfitting due to dataset size; validation accuracy

	Images		stagnates, leading to ineffective generalisation
VGG19	Cross Correlation 2D Images	50.00	Severe underfitting due to dataset size; validation accuracy stagnates, leading to ineffective generalisation
VGG19	Contrast Correlation 2D Images	50.00	Severe underfitting due to dataset size; validation accuracy stagnates, leading to ineffective generalisation
ResNet-18	**Optical Flow 2D Images**	**65.38**	**Modest generalisation**
ResNet-18	Optical Flow Spectrogram 2D Images	53.85	Severe underfitting due to dataset size; validation accuracy stagnates, leading to ineffective generalisation
ResNet-18	Cross Correlation 2D Images	61.54	Modest generalisation
ResNet-18	Contrast Correlation 2D	61.54	Modest generalisation

CHAPTER 5 Comparative Analysis and Discussion

This section presents and critically discusses the results from the classification tasks, comparing traditional machine learning algorithms and deep learning architectures across three signal preprocessing techniques: Optical Flow, Cross Correlation, and Contrast Correlation. The aim was to determine how different representations of cerebral blood flow, extracted from laser speckle imaging, influence the ability of various models to distinguish between resting and visually stimulated cognitive states.

5.1 Signal Trends in Processed Outputs

As seen in figures (4.1-4.7) Each processing technique produced scalar time-series signals that conveyed temporal dynamics of cerebral blood flow (CBF). When visual stimulus was introduced, a consistent increase in signal amplitude was observed across all three techniques, particularly Optical Flow, which produced more distinguishable differences between resting and stimulated segments (see Figures 4.1 and 4.2). These patterns suggest an increase in blood flow within the occipital lobe, confirming the hypothesis that CBF rises during increased visual processing. Some trials, however, exhibited anomalies such as sudden peaks or irregular oscillations, which are likely attributed to motion artefacts or optical feedback interference. These artefacts may have contributed to classifier mislabelling in later stages, especially when the signal representation was noisy or ambiguous.

5.2 Classifier Comparisons

5.2.1 Interpreting Confusion Matrices Beyond Accuracy

While accuracy served as the central benchmark for comparison, a deeper understanding of classifier performance requires evaluating the balance between sensitivity and specificity, particularly in biomedical contexts where false positives and false negatives carry different implications.

Across all classifier configurations and preprocessing pipelines, each output was assessed using confusion matrices. These matrices serve as the foundation for calculating a variety of diagnostic performance metrics, including accuracy, precision, recall (sensitivity), specificity, and negative predictive value (NPV).

In this investigation, visual stimulation was treated as the positive class, making recall (sensitivity) and precision directly relevant to its detection. For example, classifiers with high recall ensure that cognitive stimulation is reliably detected, reducing the chance of missing critical events (low FN). Conversely, models with high specificity minimise the misclassification of resting states as stimulation, avoiding false alarms (low FP).

Accuracy, the proportion of total correct classifications (TP + TN) out of all predictions, was used as the primary comparison metric between classifiers. However, accuracy alone does not capture the full picture, particularly in a balanced dataset like this one, where each class (rest and stimulus) is represented equally. To that end, incorporating precision (TP / [TP + FP]) and recall (TP / [TP + FN]) is crucial when discussing how well models identify states of visual stimulation (the positive class).

- A high precision score means the classifier rarely misclassifies rest as stimulation (low FP).
- A high recall score means it rarely misses stimulation when it is present (low FN).
- Specificity (TN / [TN + FP]) assesses how well a model avoids false alarms during resting segments.

For instance, the SVM model trained on contrast correlation not only achieved 94.60% accuracy, but also produced a perfect recall rate, with zero false negatives. This implies the model never missed a visually stimulated state, an essential trait for any neuroimaging-based decision system. Another example, Optical Flow and Random Forest Combined had the following results; 32.43% TN and 37.84% TP, with 10.81% FP and 18.92% FN. This shows a moderate performance but slightly more false negatives, implying some under detection of stimulus segments.

CNNs, in contrast, were less effective across all input formats, achieving no more than 65.38% accuracy even under optimal configurations (ResNet-18 with Optical Flow). Performance metrics suggest overfitting, and confusion matrices showed poor specificity and high false positive rates, which are all critical weaknesses when applying this in a real-time diagnostic setting.

5.2.2 Support Vector Machine (SVM)

For Support Vector Machines, contrast correlation paired with an RBF kernel led to near-perfect classification (94.60%), suggesting highly linearly separable features with minimal noise. This aligns with the expectation that contrast correlation provides a robust temporal representation of signal similarity, and its filtering procedure (10th-order Butterworth, 0.02–3 Hz) effectively suppresses non-informative frequencies. Conversely, Optical Flow and Cross Correlation performed best under a linear kernel, indicating less curvature or complexity in the extracted features.

The differences in results for Support Vector Machine are attributed to how each method encodes cerebral blood flow (CBF) dynamics. Contrast Correlation emphasises global structural changes in pixel intensity, reducing noise and better capturing systemic haemodynamic shifts linked to cognitive states. Even though Optical Flow and Cross Correlation created precise spatial detail, this in fact hindered SVM's ability to classify brain states. Increasing feature complexity makes linear separability harder, which is essential for SVM effectiveness without more advanced kernels or dimensionality reductions.

5.2.3 Random Forest (RF)

Random Forest classifiers showed consistent performance, with Contrast Correlation again emerging as the best pairing, yielding 89.19% accuracy. Both Cross and Optical Flow yielded lower accuracy values (62.16% and 70.27%, respectively), due in part to the higher rate of false negatives (stimulated segments misclassified as rest) and false positives.

The tree-based structure and ensemble nature of RF contributed to its superior generalisation capacity in these contexts. This suggests that the hierarchical structure of Random Forest was able to effectively capture variance introduced by contrast-based temporal changes, while also being more resilient to outliers or anomalies than SVM in noisier representations like cross-correlation.

5.2.4 VGG19

VGG19 produced uniformly low accuracy (~50%) across all processing pipelines, with

flat training curves and minimal learning, as evidenced in Figure 5.1.

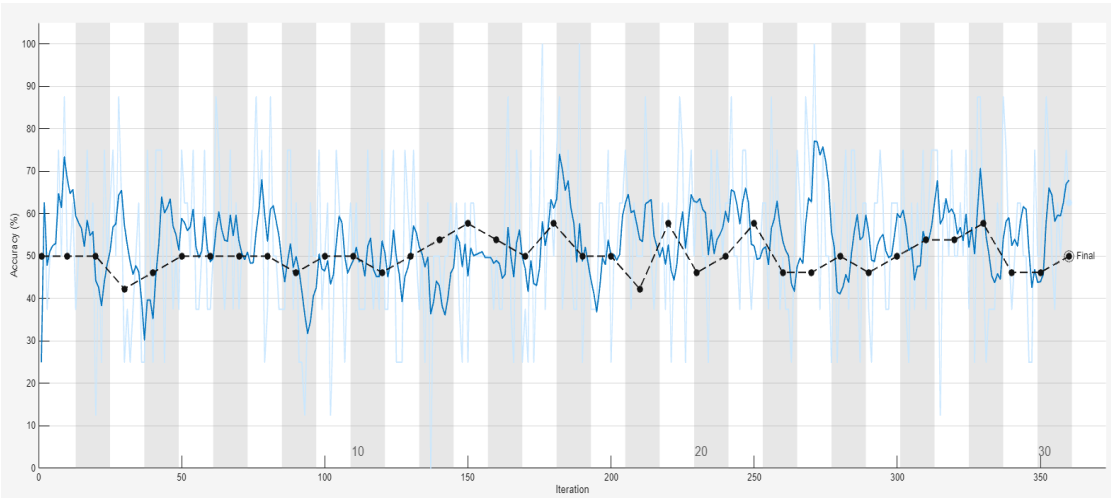


Figure 5.1: VGG19 Training Curve when paired with contrast correlation images.

Figure 5.1 shows that the training accuracy fluctuates heavily while the validation accuracy remains flat around 50%, indicating the model struggled to generalise, likely due to over-parameterisation and insufficient training data for a model of VGG19's depth.

This result reinforces the hypothesis that deeper CNNs like VGG19 are prone to overfitting on small, low-variance datasets like those in this study. With 138 million parameters, VGG19 lacked the ability to generalise from such limited training data, unlike ResNet-18's more compact architecture.

5.2.5 ResNet-18

ResNet-18 demonstrated good generalisation, achieving a peak accuracy of 65.38% on Optical Flow images and 61.54% for both Cross and Contrast Correlation. The training curve for ResNet18 (Figure 5.2) shows high training accuracy but fluctuating validation accuracy, indicative of overfitting, likely due to the small sample size.

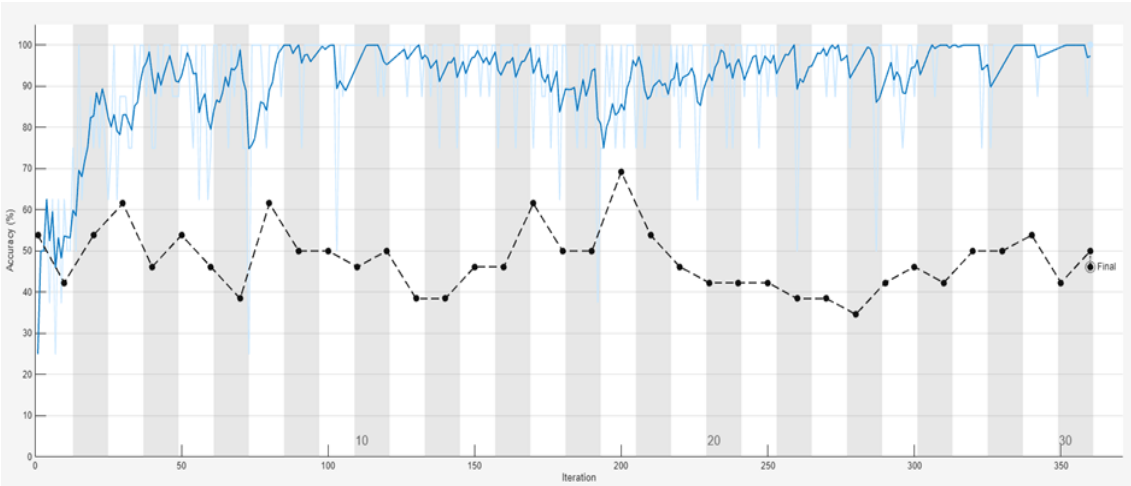


Figure 5.2: ResNet18 Training Curve when paired with contrast correlation images.

Figure 5.2 shows that training accuracy quickly approaches 100%, while validation accuracy stabilises around 60–70%, indicating successful learning but with minor overfitting due to limited dataset size.

5.3 Future Work

While this project successfully demonstrated the feasibility of using laser speckle data to classify cognitive brain states, it also exposed limitations and areas for future improvement.

A key objective of this study was to evaluate whether convolutional neural networks, which are traditionally designed for large-scale image data, could perform reliably on a comparatively small dataset. However, the results show that CNNs, by design, require far more data than what was available here to fully realise their potential. It is, therefore, somewhat unfair to benchmark CNNs against traditional machine learning algorithms within this context. The CNN's poor generalisation and overfitting, particularly in the VGG19 results, were not necessarily due to model design flaws, but rather a mismatch between architecture complexity and dataset size. Deep learning models like VGG19 and ResNet18 contain millions of parameters. These parameters are under-constrained without sufficient data, leading the model to memorise noise instead of learning generalisable patterns.

The CNN models failed to learn meaningful decision functions despite employing data

visualisation via line plots and spectrograms. Even with pre-trained architectures like VGG19 and ResNet-18, the image-based approach underperformed traditional vector-based methods, where engineered features provided clear separability. The spectrogram input (tested due to its success in audio and biomedical time-series tasks) did not improve performance. VGG19 and ResNet-18 both achieved exactly 50% classification accuracy, which is no better than blind guessing despite the spectrogram's rich frequency representation. This suggests that image input format alone cannot compensate for dataset size and signal complexity.

In terms of model engineering, multimodal fusion represents another avenue of advancement. For example, combining feature sets extracted from both optical flow and contrast correlation could lead to more powerful models. The contrast-based pipeline already demonstrated exceptionally high accuracy with both Random Forest (89.19%) and SVM (94.60%) classifiers. Fusing it with another signal stream, like optical flow, would give a more nuanced, redundant signal that improves classification confidence.

To translate these models for real-world use, applying techniques like SHapley Additive exPlanations and Local Interpretable Model-Agnostic Explanations would give insight as to what features influence the model’s prediction. For contrast-SVM configuration, there are already promising results that can be used advantageously in the field of neuroimaging. However, because of how high-stakes the field is, showing the inner workings of these models would give more credence and trust in the use of Artificial Intelligence within the medical field.

5.4 Implications for Artificial Intelligence in Medical Imaging

The broader implications of this work lie in its relevance to AI in neuroimaging. Classifying brain states such as visual engagement versus rest is not simple for a model; these are subtle haemodynamic shifts, not overt structural changes. Models that can consistently maintain a high level of accuracy are in demand because of what they command from the model with little to no errors if possible. It must be robust to noise, sensitive to weak patterns, and most importantly, accurate enough to avoid serious misclassifications.

The diagnostic confusion matrix framework, specifically true positives, false positives,

false negatives and true negatives, is essential to evaluating clinical safety. A high false negative rate (low recall) would imply that a subject is engaged, but the system fails to detect it, potentially leading to underestimations in neural responsiveness. This could be damaging in closed-loop feedback systems or Brain-Computer Interface (BCI) applications where timely detection of cognitive states is required for intervention. On the other hand, a high false positive rate (low specificity) could create the illusion of engagement where none exists, resulting in misguided interpretations, poor feedback loops, and potentially unnecessary clinical action. Precision and specificity matter the same as overall accuracy. Where the system goes wrong is just as important as how the system is correct.

From a technical standpoint, the standout performance of Contrast Correlation-based models suggests a breakthrough:

- The results confirm that a discernible pattern exists in the data, indicating that the underlying cerebral blood flow changes captured by contrast correlation are not random.
- Both Random Forest and SVM models achieved accuracies over 89% and 94%, respectively, using the same features. This is significant and therefore cannot be considered a coincidence. It validates the signal quality and demonstrates that traditional ML can be further refined on this basis, possibly for real-world applications.

This opens exciting avenues not only for classification of brain states but for real-time monitoring of cognitive workload, fatigue, or attention span; domains that are actively being explored in human-computer interaction, educational technologies, and even mental health monitoring.

In future implementations, models like these can have a significant role in clinical diagnosis or in adaptive therapeutic interventions, where AI systems adjust treatment based on inferred brain states. To reach this further validation on larger datasets and more diverse populations needs to be done, and explainability must remain a priority if these models are to be trusted by clinicians.

CHAPTER 6 CONCLUSION

This project investigated whether cerebral blood flow, captured with laser speckle imaging (LSI), could be used to classify cognitive brain states, resting versus visually stimulated, using both traditional and deep learning approaches. It also examined the viability of convolutional neural networks on a deliberately small dataset.

All objectives were met: raw speckle data were pre-processed using Optical Flow, Cross Correlation, and Contrast Correlation, and features were extracted via PCA and Relief-F. These were then classified using Support Vector Machines, Random Forests, and convolutional neural networks. Evaluation was conducted using standard metrics, including accuracy and recall.

Support Vector Machine trained on contrast correlation achieved the highest performance, with an accuracy of 94.60%, while Random Forests reached 89.19%, demonstrating that traditional classifiers paired with contrast correlation are highly effective. In contrast, convolutional neural networks underperformed, rarely exceeding 65%, confirming that such models are unsuitable for small datasets and prone to overfitting without sufficient input data.

Limitations include the small participant pool and limited stimulus diversity. While contrast correlation proved effective in this setting, its generalisability to other cognitive tasks and conditions remains untested.

This study contributes a novel and replicable framework for non-contact, machine learning-based neuroimaging. It identifies contrast correlation as a computationally efficient and discriminative signal processing technique and lays the groundwork for future developments in explainable AI, multimodal fusion, and brain-computer interface applications.

CHAPTER 7 REFERENCES

1. Laser Speckle Contrast Imaging of Cerebral Blood Flow | Annals of Biomedical Engineering [Internet]. [cited 2025 Apr 7]. Available from: https://link.springer.com/article/10.1007/s10439-011-0469-0

2. Fercher AF, Briers JD. Flow visualization by means of single-exposure speckle photography. Opt Commun. 1981 Jun 1;37(5):326–30.

3. Linkous C, Pagan AD, Shope C, Andrews L, Snyder A, Ye T, et al. Applications of Laser Speckle Contrast Imaging Technology in Dermatology. JID Innov. 2023 Sep 1;3(5):100187.

4. SpinalCord.com. Occipital Lobe: Function, Location, and Structure [Internet]. [cited 2025 Apr 7]. Available from: https://www.spinalcord.com/occipital-lobe

5. Scholkmann F, Kleiser S, Metz AJ, Zimmermann R, Mata Pavia J, Wolf U, et al. A review on continuous wave functional near-infrared spectroscopy and imaging instrumentation and methodology. NeuroImage. 2014 Jan 15;85:6–27.

6. Kassab A, Lan JL, Vannasing P, Sawan M. Functional near-infrared spectroscopy caps for brain activity monitoring: a review. Appl Opt. 2015 Jan 20;54(3):576–86.

7. Gervain J, Minagawa Y, Emberson L, Lloyd-Fox S. Using functional near-infrared spectroscopy to study the early developing brain: future directions and new challenges. Neurophotonics. 2023 Apr;10(2):023519.

8. Eggebrecht AT, White BR, Ferradal SL, Chen C, Zhan Y, Snyder AZ, et al. A quantitative spatial comparison of high-density diffuse optical tomography and fMRI cortical mapping. NeuroImage. 2012 Jul 16;61(4):1120–8.

9. The Evolution of Data Processing: A Journey Through Time [Internet]. [cited 2025 Apr 7]. Available from: https://www.linkedin.com/pulse/evolution-data-processing-journey-through-time-abhishant-gautam-n5owc

10. What is Structured Query Language (SQL)? | IBM [Internet]. 2024 [cited 2025 Apr 7]. Available from: https://www.ibm.com/think/topics/structured-query-language

11. OpenCV: Optical Flow [Internet]. [cited 2025 Apr 9]. Available from: https://docs.opencv.org/3.4/d4/dee/tutorial_optical_flow.html

12. Nanonets Blog | AI Document Processing & Workflow Automation [Internet]. 2019 [cited 2025 Apr 9]. Motion Estimation with Optical Flow: A Comprehensive Guide. Available from: https://nanonets.com/blog/optical-flow/

13. Boesch G. viso.ai. 2024 [cited 2025 Apr 9]. Optical Flow - Everything You Need to Know in 2025. Available from: https://viso.ai/deep-learning/optical-flow/

14. Farnebäck G. Two-Frame Motion Estimation Based on Polynomial Expansion. In: Bigun J, Gustavsson T, editors. Image Analysis. Berlin, Heidelberg: Springer Berlin

Heidelberg; 2003. p. 363–70.

15. jball. What is cross-correlation, and how does it advance spectrum analysis? [Internet]. Liquid Instruments. 2024 [cited 2025 Apr 8]. Available from: https://liquidinstruments.com/blog/cross-correlation-and-spectrum-analysis/

16. Gonzalez RC, Woods RE. Digital image processing. Fourth edition, global edition. New York: Pearson; 2017. 1 p.

17. Foote KD. DATAVERSITY. 2021 [cited 2025 Apr 7]. A Brief History of Machine Learning. Available from: https://www.dataversity.net/a-brief-history-of-machine-learning/

18. Deep Learning vs. Machine Learning – What's The Difference? [Internet]. [cited 2025 Apr 7]. Available from: https://levity.ai/blog/difference-machine-learning-deep-learning

19. Difference in Data Mining Vs Machine Learning Vs Artificial Intelligence [Internet]. [cited 2025 Apr 7]. Available from: https://www.softwaretestinghelp.com/data-mining-vs-machine-learning-vs-ai/

20. Neural Networks - History [Internet]. [cited 2025 Apr 7]. Available from: https://cs.stanford.edu/people/eroberts/courses/soco/projects/neural-networks/History/history1.html

21. Alan Turing Publishes 'On Computable Numbers,' Describing What Came to be Called the 'Turing Machine' : History of Information [Internet]. [cited 2025 Apr 7]. Available from: https://www.historyofinformation.com/detail.php?entryid=735

22. MIT News | Massachusetts Institute of Technology [Internet]. 2017 [cited 2025 Apr 7]. Explained: Neural networks. Available from: https://news.mit.edu/2017/explained-neural-networks-deep-learning-0414

23. What is a Neural Network? | IBM [Internet]. 2021 [cited 2025 Apr 7]. Available from: https://www.ibm.com/think/topics/neural-networks

24. Brownlee J. How Much Training Data is Required for Machine Learning? [Internet]. MachineLearningMastery.com. 2017 [cited 2025 Apr 7]. Available from: https://www.machinelearningmastery.com/much-training-data-required-machine-learning/

25. How Much Data Is Required for Machine Learning? – PostIndustria [Internet]. [cited 2025 Apr 7]. Available from: https://postindustria.com/how-much-data-is-required-for-machine-learning/

26. AI Algorithms: Traditional Machine Learning vs. Deep Learning - Cow-Shed Startup [Internet]. [cited 2025 Apr 7]. Available from: https://www.cow-shed.com/blog/ai-algorithms-traditional-machine-learning-vs-deep-learning

27. Springboard Blog [Internet]. 2021 [cited 2025 Apr 7]. Regression vs. Classification in Machine Learning: What's the Difference? Available from:

https://www.springboard.com/blog/data-science/regression-vs-classification/

28. Simplilearn.com [Internet]. 2022 [cited 2025 Apr 8]. Regression vs. Classification in Machine Learning for Beginners | Simplilearn. Available from: https://www.simplilearn.com/regression-vs-classification-in-machine-learning-article

29. Berwick R. An Idiot's guide to Support vector machines (SVMs).

30. All You Need to Know About Support Vector Machines [Internet]. Spiceworks Inc. [cited 2025 Apr 8]. Available from: https://www.spiceworks.com/tech/big-data/articles/what-is-support-vector-machine/

31. Support Vector Machine (SVM) in 7 minutes - Fun Machine Learning [Internet]. 2017 [cited 2025 Apr 8]. Available from: https://www.youtube.com/watch?v=Y6RRHw9uN9o

32. Support Vector Machine Explained-Theory, Implementation, and Visualization [Internet]. [cited 2025 Apr 8]. Available from: https://www.linkedin.com/pulse/support-vector-machine-explained-theory-visualization-zixuan-zhang

33. scikit-learn [Internet]. [cited 2025 Apr 8]. 1.11. Ensembles: Gradient boosting, random forests, bagging, voting, stacking. Available from: https://scikit-learn/stable/modules/ensemble.html

34. What Is Random Forest? | IBM [Internet]. 2021 [cited 2025 Apr 8]. Available from: https://www.ibm.com/think/topics/random-forest

35. Sruthi. Understanding Random Forest Algorithm With Examples [Internet]. Analytics Vidhya. 2021 [cited 2025 Apr 8]. Available from: https://www.analyticsvidhya.com/blog/2021/06/understanding-random-forest/

36. Eland M. Matt on ML.NET. 2022 [cited 2025 Apr 19]. Interpreting Confusion Matrixes. Available from: https://accessibleai.dev/post/interpreting_confusion_matrixes/

37. Ozana N, Noah JA, Zhang X, Ono Y, Hirsch J, Zalevsky Z. Remote photonic sensing of cerebral hemodynamic changes via temporal spatial analysis of acoustic vibrations. J Biophotonics. 2020;13(2):e201900201.

38. Kalyuzhner Z, Agdarov S, Orr I, Beiderman Y, Bennett A, Zalevsky Z. Remote photonic detection of human senses using secondary speckle patterns. Sci Rep. 2022 Jan 11;12(1):519.

39. Torricelli A, Contini D, Mora AD, Pifferi A, Re R, Zucchelli L, et al. Neurophotonics: non-invasive optical techniques for monitoring brain functions. Funct Neurol. 2014 Dec 3;29(4):223–30.

40. Villringer A, Chance B. Non-invasive optical spectroscopy and imaging of human brain function. Trends Neurosci. 1997 Oct 1;20(10):435–42.

41. McIntosh AR, Cabeza RE, Lobaugh NJ. Analysis of neural interactions explains the activation of occipital cortex by an auditory stimulus. J Neurophysiol. 1998 Nov;80(5):2790–6.

42. Du E, Zheng H, He H, Li S, Qiu C, Zhang W, et al. Dual-Wavelength Confocal Laser Speckle Contrast Imaging Using a Deep Learning Approach. Photonics. 2024 Nov;11(11):1085.

43. Hong J, Zhu W, He K, Chen X, Lu J, Li P. Ergodic speckle contrast optical coherence tomography velocimetry of rapid blood flow. Opt Lett. 2024 Jul 1;49(13):3600–3.

44. Tong Y, Hocke LM, Frederick BB. Low Frequency Systemic Hemodynamic "Noise" in Resting State BOLD fMRI: Characteristics, Causes, Implications, Mitigation Strategies, and Applications. Front Neurosci [Internet]. 2019 Aug 14 [cited 2025 Apr 9];13. Available from: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2019.00787/full

45. Hussain SAH, Raza I, Hussain SA, Jamal MH, Gulrez T, Zia A. A mental state aware brain computer interface for adaptive control of electric powered wheelchair. Sci Rep. 2025 Mar 22;15(1):9880.

46. Horn B, Schunck B. Determining Optical Flow. Artif Intell. 1981 Aug 1;17:185–203.

47. Parthasarathy AB, Kazmi SMS, Dunn AK. Quantitative imaging of ischemic stroke through thinned skull in mice with Multi Exposure Speckle Imaging. Biomed Opt Express. 2010 Jul 15;1(1):246–59.

48. Briers D. Laser Speckle Contrast Imaging for Measuring Blood Flow. Opt Appl. 2007 Jan 1;37.

49. Sohil F, Sohail M, Shabbir J. An introduction to statistical learning with applications in R: by Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani, New York, Springer Science and Business Media, 2013, $41.98, eISBN: 978-1-4614-7137-7. Stat Theory Relat Fields. 2021 Sep 26;6:1–1.

50. Get Started with Transfer Learning [Internet]. [cited 2025 Apr 9]. Available from: https://uk.mathworks.com/help/deeplearning/gs/get-started-with-transfer-learning.html

51. Resnet50 Number Of Parameters | Restackio [Internet]. [cited 2025 Apr 19]. Available from: https://www.restack.io/p/resnet50-answer-number-of-parameters-cat-ai

52. Domingos P. A few useful things to know about machine learning. Commun ACM. 2012 Oct;55(10):78–87.

53. Anwar A. Difference between AlexNet, VGGNet, ResNet and Inception [Internet]. TDS Archive. 2022 [cited 2025 Apr 19]. Available from: https://medium.com/data-

science/the-w3h-of-alexnet-vggnet-resnet-and-inception-7baaaecccc96

54. #013 CNN VGG 16 and VGG 19 – Master Data Science [Internet]. [cited 2025 Apr 19]. Available from: https://datahacker.rs/deep-learning-vgg-16-vs-vgg-19/

55. Fox PT, Raichle ME. Stimulus rate dependence of regional cerebral blood flow in human striate cortex, demonstrated by positron emission tomography. J Neurophysiol. 1984 May;51(5):1109–20.

CHAPTER 8 APPENDIX A CODE

SVM (OPTICAL/CROSS/CONTRAST)

```
filePrefix = '20231027';
fileNumbers = 1:21;
fps = 1550;
duration_in_sec = 90;
Nframes = duration_in_sec * fps;
opticFlow = opticalFlowFarneback;
f_sample = 1550;
low_stop = 0.02;
high_stop = 3;
t = (0:Nframes - 1) / f_sample;
% Struct for storing brain signal results
brainsignal = struct('xpos', [], 'ypos', [], 'X', [], 'Y', []);
resultsArray = cell(1, numel(fileNumbers));
maxFrames = 0;


for idx = 1:numel(fileNumbers)
    fileNumber = fileNumbers(idx);
    fileName = sprintf('%s_%d.hdf5', filePrefix, fileNumber);
    h5info(fileName);
    frames = h5read(fileName, '/24505982');  % Load frames from HDF5 file
    Nframes = size(frames, 3);
    xpos = zeros(1, Nframes);
    ypos = zeros(1, Nframes);


    % Calculate Optical Flow for Each Frame
    for i = 1:Nframes
        flow = estimateFlow(opticFlow, rescale(squeeze(frames(:, :, i)),
0, 1));
        flMA = flow.Magnitude / 100;
        flMP = flow.Orientation;
        vect = sum(sum(flMA .* exp(1j * flMP)));
        xpos(i + 1) = xpos(i) + real(vect);
        ypos(i + 1) = ypos(i) + imag(vect);
    end
```

```
    resultsArray{idx} = struct('xpos', xpos, 'ypos', ypos);
    maxFrames = max(maxFrames, Nframes);
end


% Filtering the Signals
brainsignal_xpos = zeros(numel(fileNumbers), maxFrames);
brainsignal_ypos = zeros(numel(fileNumbers), maxFrames);
brainsignal_X = zeros(numel(fileNumbers), maxFrames);
brainsignal_Y = zeros(numel(fileNumbers), maxFrames);


for i = 1:numel(fileNumbers)
    xpos = resultsArray{i}.xpos;
    ypos = resultsArray{i}.ypos;
    brainsignal_xpos(i, 1:numel(xpos)) = xpos;
    brainsignal_ypos(i, 1:numel(ypos)) = ypos;

    % Bandpass Filter Design
    d_st = fdesign.bandpass('N,F3dB1,F3dB2', 10, low_stop, high_stop,
f_sample);
    Hd_st = design(d_st, 'butter');

    % Apply Filter
    x_filtered = filter(Hd_st, xpos);
    y_filtered = filter(Hd_st, ypos);
    brainsignal_X(i, 1:numel(x_filtered)) = x_filtered;
    brainsignal_Y(i, 1:numel(x_filtered)) = y_filtered;
end


combine_brainsignal = sqrt(brainsignal_X.^2 + brainsignal_Y.^2);
combine_final = combine_brainsignal(:, 1:end - 1);
num_samples = size(combine_final, 1);
num_intervals = 6;
interval_length = 15;
sample_rate = 1550;
sample_length_per_interval = interval_length * sample_rate;


relaxingdata = zeros(sample_length_per_interval, num_samples *
num_intervals);
payingattentiondata = zeros(sample_length_per_interval, num_samples *
num_intervals);


for i = 1:num_samples
    current_sample = combine_final(i, :);
    for j = 1:num_intervals
        start_idx = (j - 1) * sample_length_per_interval + 1;
```

```
        end_idx = min(j * sample_length_per_interval,
length(current_sample));
        interval_data = current_sample(start_idx:end_idx);
        if mod(j, 2) == 1
            relaxingdata(1:length(interval_data), (i - 1) * num_intervals
+ j) = interval_data;
        else
            payingattentiondata(1:length(interval_data), (i - 1) *
num_intervals + j) = interval_data;
        end
    end
end


% Data Preparation and Plotting
relaxingdata = relaxingdata';
payingattentiondata = payingattentiondata';
relaxingdata = relaxingdata(:, any(relaxingdata));
payingattentiondata = payingattentiondata(:, any(payingattentiondata));
% Remove rows with all zero values from relaxingdata and
payingattentiondata
relaxingdata(~any(relaxingdata, 2), :) = [];
payingattentiondata(~any(payingattentiondata, 2), :) = [];


% Plotting the first six sample
figure;
for i = 1:min(6, size(relaxingdata, 1))
    subplot(6, 1, i);
    plot(relaxingdata(i, :));
    title(['Relaxing Data Sample ' num2str(i)]);
    xlabel('Time (samples)');
    ylabel('Amplitude');
end


figure;
for i = 1:min(6, size(payingattentiondata, 1))
    subplot(6, 1, i);
    plot(payingattentiondata(i, :));
    title(['Paying Attention Data Sample ' num2str(i)]);
    xlabel('Time (samples)');
    ylabel('Amplitude');
end


% Calculating Means and Standard deviation
mean_relaxing = mean(relaxingdata(:));
std_relaxing = std(relaxingdata(:));
mean_payingattention = mean(payingattentiondata(:));
std_payingattention = std(payingattentiondata(:));
```

```
fprintf('Relaxing Data - Mean: %.4f, Std: %.4f\n', mean_relaxing,
std_relaxing);
fprintf('Paying Attention Data - Mean: %.4f, Std: %.4f\n',
mean_payingattention, std_payingattention);


% Extracting feat
mldata = cat(1, payingattentiondata, relaxingdata);
labels = [ones(size(payingattentiondata, 1), 1); zeros(size(relaxingdata,
1), 1)];
mean_feature = mean(mldata, 2);
mean_sq_feature = mean(mldata.^2, 2);
std_feature = std(mldata, 0, 2);
variance_feature = var(mldata, 0, 2);
lower_feature = prctile(mldata, 25, 2);
upper_feature = prctile(mldata, 75, 2);
fft_feature = fft(mldata, [], 2);
power_spect_feature = abs(fft_feature).^2 / 23250;
mag_fft_feature = abs(fft_feature);
mean_mag_fft_feat = mean(mag_fft_feature, 2);
tot_power_feat = sum(power_spect_feature.^2, 2);


% Feature extraction
num_features = min(size(mldata, 1), 126);
corr_val_feat = zeros(num_features, 1);
peaks_troughs_feat = cell(num_features, 2);
mean_freq_feat = zeros(num_features, 1);
mean_phase_feat = zeros(num_features, 1);
fft_coeff_feat = zeros(num_features, 3);
wavelet_features = zeros(num_features, 4);


for i = 1:num_features
    corr_val_feat(i) = mean(xcorr(mldata(i, :), mldata(i, :), 'coeff'));
    [peak_vals, peaks] = findpeaks(power_spect_feature(i, :));
    [troughs_vals, troughs] = findpeaks(-power_spect_feature(i, :));
    peaks_troughs_feat{i, 1} = max(peak_vals, [], 'omitnan');
    peaks_troughs_feat{i, 2} = min(-troughs_vals, [], 'omitnan');
    freq = (0:23250 - 1) * (sample_rate / 89);
    mean_freq_feat(i) = sum(power_spect_feature(i, :) .* freq) /
sum(power_spect_feature(i, :));
    phase_feat = angle(fft(mldata(i, :)));
    mean_phase_feat(i) = mean(phase_feat);
    fft_coeff_feat(i, 1) = mean(mldata(i, :));
    [C, ~] = wavedec(mldata(i, :), 3, 'db1');
    wavelet_features(i, 1) = mean(C);
    wavelet_features(i, 2) = std(C);
    wavelet_features(i, 3) = var(C);
```

```
    coeff = hist(C, linspace(min(C), max(C), 100)) / numel(C);
    wavelet_entropy = -sum(coeff .* log2(coeff + eps));
    wavelet_features(i, 4) = wavelet_entropy;
end


skewness_feature = skewness(mldata, 0, 2);
kurtosis_feature = kurtosis(mldata, 0, 2);
third_moment_feature = moment(mldata, 3, 2);
gradient_values = gradient(mldata, 1, 2);
gradient_feature = sqrt(sum(gradient_values.^2, 2));
pt_feature = cell2mat(peaks_troughs_feat);
features = cat(2, mean_feature, std_feature, variance_feature,
upper_feature, lower_feature, mean_mag_fft_feat, corr_val_feat,
pt_feature, mean_freq_feat, mean_sq_feature, mean_phase_feat,
fft_coeff_feat, wavelet_features, skewness_feature, kurtosis_feature,
third_moment_feature, gradient_feature);
mu_features = mean(features(:));
sigma_features = std(features(:));
normalised_features = 2 * (features - mu_features) / sigma_features;


% PCA Feature Reduction
[coefficients, score, ~, ~, explained] = pca(mldata);
num_components = find(cumsum(explained) >= 95, 1);
pca_final = score(:, 1:num_components);
[scores, ranking] = relieff(features, labels, 2);
num_features = 23;
selected_features1 = ranking(1:num_features);
cv = cvpartition(size(features, 1), 'HoldOut', 0.3);
train_idx = training(cv);
test_idx = test(cv);


training_data = features(train_idx, :);
testing_data = features(test_idx, :);
training_labels = labels(train_idx);
testing_labels = labels(test_idx);


svm_model = fitcsvm(training_data, training_labels, ...
                    'KernelFunction', 'linear', ...
                    'Standardize', true, ...
                    'ClassNames', [0, 1], ...
                    'KernelScale', 'auto');


predicted_labels = predict(svm_model, testing_data);
accuracy = sum(predicted_labels == testing_labels) /
numel(testing_labels);
```

```
disp(['Holdout Test Accuracy (70/30 split): ' num2str(accuracy * 100)
'%']);


% Confusion matrix (valid - from holdout test set)
C = confusionmat(testing_labels, predicted_labels);
totalsamples = sum(C(:));
percentages = (C / totalsamples) * 100;


figure;
h = heatmap(percentages, 'Colormap', jet, 'ColorLimits', [0, 100], ...
            'FontSize', 10, 'XDisplayLabels', {'0', '1'}, ...
            'YDisplayLabels', {'0', '1'});
title('OpticalFlow - SVM');
xlabel('Predicted Labels');
ylabel('True Labels');


fprintf('Total number of test samples: %d\n', totalsamples);

saveas(gcf, 'opticalflow_svm_confusion_matrix.png');
```

This is SVM framework to apply cross correlation instead of optical flow replace;

```
    for i = 1:Nframes
        flow = estimateFlow(opticFlow, rescale(squeeze(frames(:, :, i)),
0, 1));
        flMA = flow.Magnitude / 100;
        flMP = flow.Orientation;
        vect = sum(sum(flMA .* exp(1j * flMP)));
        xpos(i + 1) = xpos(i) + real(vect);
        ypos(i + 1) = ypos(i) + imag(vect);
    end
    resultsArray{idx} = struct('xpos', xpos, 'ypos', ypos);
    maxFrames = max(maxFrames, Nframes);
end
```

with

```
corrVec = zeros(1, Nframes - 1);


for i = 2:Nframes
    f1 = double(frames(:, :, i - 1));
    f2 = double(frames(:, :, i));
    R = xcorr2(f1 - mean(f1(:)), f2 - mean(f2(:)));
    corrVec(i - 1) = max(R(:));
end
```

```
% Store in the same
resultsArray{idx} = struct('xpos', corrVec, 'ypos', corrVec);


    maxFrames = max(maxFrames, Nframes);
end
```

for contrast

```
contrastCorr = zeros(1, Nframes - 1);
for i = 2:Nframes

    f1 = double(frames(:, :, i - 1));
    f2 = double(frames(:, :, i));
    f1n = (f1 - mean(f1(:))) / std(f1(:));
    f2n = (f2 - mean(f2(:))) / std(f2(:));
    contrastCorr(i - 1) = mean(f1n(:) .* f2n(:));
end
% Store contrast correlation
xpos = [0, cumsum(contrastCorr)];
ypos = zeros(1, Nframes);
```

Random Forest

This is the same for Random Forest. Below is the full code is the full code of Contrast and Random Forest. For other configurations of data processing with random forest only processing was changed.

```
% Parameters and Initialization

filePrefix = '20231027';
fileNumbers = 1:21;
fps = 1550;
duration_in_sec = 90;
Nframes = duration_in_sec * fps;
opticFlow = opticalFlowFarneback;
f_sample = 1550;
low_stop = 0.02;
high_stop = 3;
t = (0:Nframes - 1) / f_sample;


% Struct for storing
brainsignal = struct('xpos', [], 'ypos', [], 'X', [], 'Y', []);
resultsArray = cell(1, numel(fileNumbers));
maxFrames = 0;
```

```
% Load and Process Data Files
for idx = 1:numel(fileNumbers)
    fileNumber = fileNumbers(idx);
    fileName = sprintf('%s_%d.hdf5', filePrefix, fileNumber);
    h5info(fileName);
    frames = h5read(fileName, '/24505982');  % Load frames from HDF5 file
    Nframes = size(frames, 3);
    xpos = zeros(1, Nframes);
    ypos = zeros(1, Nframes);


% Calculate Contrast Correlation for Each Frame
contrastCorr = zeros(1, Nframes - 1);
for i = 2:Nframes
f1 = double(frames(:, :, i - 1));
f2 = double(frames(:, :, i));
f1n = (f1 - mean(f1(:))) / std(f1(:));
f2n = (f2 - mean(f2(:))) / std(f2(:));
contrastCorr(i - 1) = mean(f1n(:) .* f2n(:));
end
%
xpos = [0, cumsum(contrastCorr)];
ypos = zeros(1, Nframes); %
resultsArray{idx} = struct('xpos', xpos, 'ypos', ypos);
maxFrames = max(maxFrames, Nframes);
resultsArray{idx} = struct('xpos', xpos, 'ypos', ypos);
maxFrames = max(maxFrames, Nframes);
end


% Filtering the Signals
brainsignal_xpos = zeros(numel(fileNumbers), maxFrames);
brainsignal_ypos = zeros(numel(fileNumbers), maxFrames);
brainsignal_X = zeros(numel(fileNumbers), maxFrames);
brainsignal_Y = zeros(numel(fileNumbers), maxFrames);


for i = 1:numel(fileNumbers)
    xpos = resultsArray{i}.xpos;
    ypos = resultsArray{i}.ypos;
    brainsignal_xpos(i, 1:numel(xpos)) = xpos;
    brainsignal_ypos(i, 1:numel(ypos)) = ypos;

        d_st = fdesign.bandpass('N,F3dB1,F3dB2', 10, low_stop, high_stop,
f_sample);
    Hd_st = design(d_st, 'butter');

    x_filtered = filter(Hd_st, xpos);
    y_filtered = filter(Hd_st, ypos);
    brainsignal_X(i, 1:numel(x_filtered)) = x_filtered;
    brainsignal_Y(i, 1:numel(x_filtered)) = y_filtered;
```

```
end


% combine_brainsignal = sqrt(brainsignal_X.^2 + brainsignal_Y.^2);
combine_final = combine_brainsignal(:, 1:end - 1);
num_samples = size(combine_final, 1);
num_intervals = 6;
interval_length = 15;
sample_rate = 1550;
sample_length_per_interval = interval_length * sample_rate;


relaxingdata = zeros(sample_length_per_interval, num_samples *
num_intervals);
payingattentiondata = zeros(sample_length_per_interval, num_samples *
num_intervals);


for i = 1:num_samples
    current_sample = combine_final(i, :);
    for j = 1:num_intervals
        start_idx = (j - 1) * sample_length_per_interval + 1;
        end_idx = min(j * sample_length_per_interval,
length(current_sample));
        interval_data = current_sample(start_idx:end_idx);
        if mod(j, 2) == 1
            relaxingdata(1:length(interval_data), (i - 1) * num_intervals
+ j) = interval_data;
        else
            payingattentiondata(1:length(interval_data), (i - 1) *
num_intervals + j) = interval_data;
        end
    end
end


relaxingdata = relaxingdata';
payingattentiondata = payingattentiondata';
relaxingdata = relaxingdata(:, any(relaxingdata));
payingattentiondata = payingattentiondata(:, any(payingattentiondata));
relaxingdata(~any(relaxingdata, 2), :) = [];  % Remove rows where all
elements are zero
payingattentiondata(~any(payingattentiondata, 2), :) = [];  % Remove rows
where all elements are zero


% Plotting the first six samples of relaxing data
figure;
for i = 1:min(6, size(relaxingdata, 1))
    subplot(6, 1, i);
    plot(relaxingdata(i, :));
```

```
    title(['Relaxing Data Sample ' num2str(i)]);
    xlabel('Time (samples)');
    ylabel('Amplitude');
end


% Plotting the first six samples of paying attention data for i =
1:min(6, size(payingattentiondata, 1))
    subplot(6, 1, i);
    plot(payingattentiondata(i, :));
    title(['Paying Attention Data Sample ' num2str(i)]);
    xlabel('Time (samples)');
    ylabel('Amplitude');
end


mean_relaxing = mean(relaxingdata(:));
std_relaxing = std(relaxingdata(:));
mean_payingattention = mean(payingattentiondata(:));
std_payingattention = std(payingattentiondata(:));


fprintf('Relaxing Data - Mean: %.4f, Std: %.4f\n', mean_relaxing,
std_relaxing);
fprintf('Paying Attention Data - Mean: %.4f, Std: %.4f\n',
mean_payingattention, std_payingattention);


% Extracting Features for SVM Model
mldata = cat(1, payingattentiondata, relaxingdata);
labels = [ones(size(payingattentiondata, 1), 1); zeros(size(relaxingdata,
1), 1)];
mean_feature = mean(mldata, 2);
mean_sq_feature = mean(mldata.^2, 2);
std_feature = std(mldata, 0, 2);
variance_feature = var(mldata, 0, 2);
lower_feature = prctile(mldata, 25, 2);
upper_feature = prctile(mldata, 75, 2);
fft_feature = fft(mldata, [], 2);
power_spect_feature = abs(fft_feature).^2 / 23250;
mag_fft_feature = abs(fft_feature);
mean_mag_fft_feat = mean(mag_fft_feature, 2);
tot_power_feat = sum(power_spect_feature.^2, 2);


num_features = min(size(mldata, 1), 126);
corr_val_feat = zeros(num_features, 1);
peaks_troughs_feat = cell(num_features, 2);
mean_freq_feat = zeros(num_features, 1);
mean_phase_feat = zeros(num_features, 1);
fft_coeff_feat = zeros(num_features, 3);
```

```
wavelet_features = zeros(num_features, 4);


for i = 1:num_features
corr_val_feat(i) = mean(xcorr(mldata(i, :), mldata(i, :), 'coeff'));
[peak_vals, peaks] = findpeaks(power_spect_feature(i, :));
[troughs_vals, troughs] = findpeaks(-power_spect_feature(i, :));
if isempty(peak_vals)
peaks_troughs_feat{i, 1} = NaN;
else
peaks_troughs_feat{i, 1} = max(peak_vals);
end
if isempty(troughs_vals)
peaks_troughs_feat{i, 2} = NaN;
else
peaks_troughs_feat{i, 2} = min(-troughs_vals);
end
freq = (0:23250 - 1) * (sample_rate / 89);
mean_freq_feat(i) = sum(power_spect_feature(i, :) .* freq)
sum(power_spect_feature(i, :));
phase_feat = angle(fft(mldata(i, :)));
mean_phase_feat(i) = mean(phase_feat);
fft_coeff_feat(i, 1) = mean(mldata(i, :));
[C, ~] = wavedec(mldata(i, :), 3, 'db1');
wavelet_features(i, 1) = mean(C);
wavelet_features(i, 2) = std(C);
wavelet_features(i, 3) = var(C);
coeff = hist(C, linspace(min(C), max(C), 100)) / numel(C);
wavelet_entropy = -sum(coeff .* log2(coeff + eps));
wavelet_features(i, 4) = wavelet_entropy;
end
skewness_feature = skewness(mldata, 0, 2);
kurtosis_feature = kurtosis(mldata, 0, 2);
third_moment_feature = moment(mldata, 3, 2);
gradient_values = gradient(mldata, 1, 2);
gradient_feature = sqrt(sum(gradient_values.^2, 2));
pt_feature = cell2mat(peaks_troughs_feat);
features = cat(2, mean_feature, std_feature, variance_feature,
upper_feature, lower_feature, mean_mag_fft_feat, corr_val_feat,
pt_feature, mean_freq_feat, mean_sq_feature, mean_phase_feat,
fft_coeff_feat, wavelet_features, skewness_feature, kurtosis_feature,
third_moment_feature, gradient_feature);
mu_features = mean(features(:));
sigma_features = std(features(:));
normalised_features = 2 * (features - mu_features) / sigma_features;


% PCA Feature Reduction
[coefficients, score, ~, ~, explained] = pca(mldata);
num_components = find(cumsum(explained) >= 95, 1);
```

```
pca_final = score(:, 1:num_components);
[scores, ranking] = relieff(features, labels, 2);
num_features = 23;
selected_features1 = ranking(1:num_features);


% --- 70/30
cv = cvpartition(size(features, 1), 'HoldOut', 0.3);
train_idx = training(cv);
test_idx = test(cv);


training_data = features(train_idx, :);
testing_data = features(test_idx, :);
training_labels = labels(train_idx);
testing_labels = labels(test_idx);




% RANDOM FOREST CLASSIFIER
rf_model = TreeBagger(500, training_data, training_labels, ...
                      'Method', 'classification', ...
                      'OOBPrediction', 'on', ...
                      'NumPredictorsToSample', 'all', ...
                      'MinLeafSize', 5);
% Predict on test data
predicted_labels = predict(rf_model, testing_data);
predicted_labels = str2double(predicted_labels);  % Convert


% Accuracy of model
accuracy = sum(predicted_labels == testing_labels) /
numel(testing_labels);
disp(['Random Forest Test Accuracy (80/20 split): ' num2str(accuracy *
100) '%']);


% Confusion matrix
C = confusionmat(testing_labels, predicted_labels);
totalsamples = sum(C(:));
percentages = (C / totalsamples) * 100;


% Plot confusion matrix for RF
figure;
h = heatmap(percentages, ...
            'Colormap', jet, ...
            'ColorLimits', [0, 100], ...
            'FontSize', 10, ...
            'XDisplayLabels', {'0', '1'}, ...
```

```
            'YDisplayLabels', {'0', '1'});
title('Contrast Correlation - Random Forest');
xlabel('Predicted Labels');
ylabel('True Labels');


fprintf('Total number of test samples: %d\n', totalsamples);
saveas(gcf, 'contrast_rf_confusion_matrix.png');
```

Image plot for quiver

```
file = '20231027_19.hdf5';
frames = h5read(file, '/24505982');
Nframes = size(frames, 3);
volunteerID = 7;
trialID = 1;


p_rest = 30000;
p_stim = 70000;
opticflow = opticalFlowFarneback;
estimateFlow(opticflow, rescale(squeeze(frames(100:end, 100:end,
p_rest)), 0, 1));
flow_rest = estimateFlow(opticflow, rescale(squeeze(frames(100:end,
100:end, p_rest + 1)), 0, 1));


estimateFlow(opticflow, rescale(squeeze(frames(100:end, 100:end,
p_stim)), 0, 1));
flow_stim = estimateFlow(opticflow, rescale(squeeze(frames(100:end,
100:end, p_stim + 1)), 0, 1));


% Plot
figure('Position', [100, 100, 1200, 600]);


%  REST
subplot(2, 3, 1);
imagesc(frames(100:end, 100:end, p_rest));
colormap('hot');
title(sprintf('Rest: Frame %d', p_rest));


subplot(2, 3, 2);
imagesc(frames(100:end, 100:end, p_rest+1));
title(sprintf('Rest: Frame %d', p_rest + 1));


subplot(2, 3, 3);
tmpx = flow_rest.Vx;
```

```
tmpy = -flipud(flow_rest.Vy);
flMA = flow_rest.Magnitude / 100;
flMP = flow_rest.Orientation;
vect = sum(sum(flMA .* exp(1j * flMP)));
maxindex = min(29, size(tmpx, 1));
quiver(tmpx(1:4:maxindex, 1:4:maxindex), tmpy(1:4:maxindex,
1:4:maxindex), 3);
hold on;
quiver(12.5, 12.5, real(vect)/100, -imag(vect)/100, 'r');
title(sprintf('Rest Flow Vectors (Frame %d → %d)', p_rest, p_rest+1));
hold off;
subplot(2, 3, 4);
imagesc(frames(100:end, 100:end, p_stim));
colormap('hot');
title(sprintf('Stimulus: Frame %d', p_stim));
subplot(2, 3, 5);
imagesc(frames(100:end, 100:end, p_stim+1));
title(sprintf('Stimulus: Frame %d', p_stim + 1));
subplot(2, 3, 6);
tmpx = flow_stim.Vx;
tmpy = -flipud(flow_stim.Vy);
flMA = flow_stim.Magnitude / 100;
flMP = flow_stim.Orientation;
vect = sum(sum(flMA .* exp(1j * flMP)));
quiver(tmpx(1:4:maxindex, 1:4:maxindex), tmpy(1:4:maxindex,
1:4:maxindex), 3);
hold on;
quiver(12.5, 12.5, real(vect)/100, -imag(vect)/100, 'r');
title(sprintf('Stimulus Flow Vectors (Frame %d → %d)', p_stim,
p_stim+1));
hold off;
```

X and Y optical flow

```
volunteersnum = 7;

filespervol = 3;
fps = 1550;  % FR
duration_sec = 90;
total_frames = fps * duration_sec;


for i = 1:volunteersnum
    figure('Name', ['Volunteer ' num2str(i)], 'NumberTitle', 'off');
    for j = 1:filespervol
        filenum = (i - 1) * filespervol + j;
        xplot = brainsignal_X(filenum, 1:end-1);
        yplot = brainsignal_Y(filenum, 1:end-1);
```

```
        subplot(2, filespervol, j);
        plot(t(1:length(xplot)), xplot);
        title(['Volunteer ' num2str(i) ', Trial ' num2str(j)]);
        xlabel('Time (s)');
        ylabel('X Displacement');


        subplot(2, filespervol, j + filespervol);
        plot(t(1:length(yplot)), yplot);
        title(['Volunteer ' num2str(i) ', Trial ' num2str(j)]);
        xlabel('Time (s)');
        ylabel('Y Displacement');
    end

end
```

IMAGE GENERATION

Optical flow

```
clc; clear; close all;


filePrefix = '20231027';
fileNumbers = 1:21;
baseFolder = 'optical_flow_plots';
relaxingFolder = fullfile(baseFolder, 'Relaxing_Data');
payingAttentionFolder = fullfile(baseFolder, 'PayingAttention_Data');
if ~exist(relaxingFolder, 'dir'); mkdir(relaxingFolder); end
if ~exist(payingAttentionFolder, 'dir'); mkdir(payingAttentionFolder);
end


interval_length_sec = 15;
num_intervals = 6;
downsample_factor = 10;


for i = 1:numel(fileNumbers)
    fileNum = fileNumbers(i);
    fileName = sprintf('%s_%d.hdf5', filePrefix, fileNum);
    if ~isfile(fileName); continue; end


    fps = (fileNum <= 18) * 1550 + (fileNum > 18) * 1470;
    framesPerInterval = interval_length_sec * fps;



    frames = h5read(fileName, '/24505982');
```

```
    frames = permute(frames, [3, 1, 2]);


    % volunteer/trial
    vID = ceil(fileNum / 3);
    tID = mod(fileNum - 1, 3) + 1;



    for intvl = 1:num_intervals
        startIdx = (intvl - 1) * framesPerInterval + 1;
        endIdx = intvl * framesPerInterval;
        if endIdx > size(frames, 1); continue; end


        segment = frames(startIdx:endIdx, :, :);
        flowMag = zeros(framesPerInterval - 1, 1);


        % Optical flow
        opticFlow = opticalFlowFarneback;
        estimateFlow(opticFlow, im2double(squeeze(segment(1, :, :))));
        for k = 2:framesPerInterval
            f = im2double(squeeze(segment(k, :, :)));
            flow = estimateFlow(opticFlow, f);
            flowMag(k-1) = mean(hypot(flow.Vx(:), flow.Vy(:)));
        end


        % Downsample
        y = downsample(flowMag, downsample_factor);
        x = linspace(0, interval_length_sec, numel(y));


        % Save path & label
        if mod(intvl, 2)
            label = 'Relaxing';
            idx = (intvl + 1) / 2;
            outFolder = relaxingFolder;
        else
            label = 'PayingAttention';
            idx = intvl / 2;
            outFolder = payingAttentionFolder;
        end
        outName = sprintf('Volunteer%d_Trial_%d_%s%d.png', vID, tID,
label, idx);
        outPath = fullfile(outFolder, outName);

end
```

```
        y = (y - min(y)) / (max(y) - min(y));


        fig = figure('Visible', 'off');
        plot(x, y, 'k', 'LineWidth', 1.2);
        axis off;


        exportgraphics(gcf, outPath, 'BackgroundColor', 'white',
'ContentType', 'image');
        close(gcf);


    end
```

Cross

```
filePrefix = '20231027';
fileNumbers = 1:21;
interval_length_sec = 15;
num_intervals = 6;
fps = 1550;
duration_in_sec = 90;
Nframes = duration_in_sec * fps;
f_sample = 1550;
low_stop = 0.02;
high_stop = 3;
d_st = fdesign.bandpass('N,F3dB1,F3dB2', 10, low_stop, high_stop,
f_sample);
Hd_st = design(d_st, 'butter');
baseFolder = 'crosscorrV2_plots';
relaxingFolder = fullfile(baseFolder, 'Relaxing_Data');
payingAttentionFolder = fullfile(baseFolder, 'PayingAttention_Data');
if ~exist(relaxingFolder, 'dir'); mkdir(relaxingFolder); end
if ~exist(payingAttentionFolder, 'dir'); mkdir(payingAttentionFolder);
end



for i = 1:numel(fileNumbers)
    fileNum = fileNumbers(i);
    fileName = sprintf('%s_%d.hdf5', filePrefix, fileNum);
    if ~isfile(fileName); continue; end


    fps = (fileNum <= 18) * 1550 + (fileNum > 18) * 1470;
    framesPerInterval = interval_length_sec * fps;
```

```
    % Load and reshape data
    frames = h5read(fileName, '/24505982');
    frames = permute(frames, [3, 1, 2]);
    % volunteer/trial
    vID = ceil(fileNum / 3);
    tID = mod(fileNum - 1, 3) + 1;


    % Loop over intervals
    for intvl = 1:num_intervals
        % Set label and file path
        if mod(intvl, 2)
            label = 'Relaxing';
            idx = (intvl + 1) / 2;
            outFolder = relaxingFolder;
        else
            label = 'PayingAttention';
            idx = intvl / 2;
            outFolder = payingAttentionFolder;
        end


        outName = sprintf('Volunteer%d_Trial_%d_%s%d.png', vID, tID,
label, idx);
        outPath = fullfile(outFolder, outName);




        % Get interval segment
        startIdx = (intvl - 1) * framesPerInterval + 1;
        endIdx = intvl * framesPerInterval;
        if endIdx > size(frames, 1); continue; end


        segment = frames(startIdx:endIdx, :, :);
        Nframes = size(segment, 1);
        corrVec = zeros(1, Nframes - 1);


        for k = 2:Nframes
            f1 = double(squeeze(segment(k - 1, :, :)));
            f2 = double(squeeze(segment(k, :, :)));
            R = xcorr2(f1 - mean(f1(:)), f2 - mean(f2(:)));
            corrVec(k - 1) = max(R(:));
        end


        % Apply bandpass filter
        corrVec_filt = filter(Hd_st, corrVec);
```

```
            y = corrVec_filt;


        x = linspace(0, interval_length_sec, numel(y));


        % Plot and save
        fig = figure('Visible', 'off');
        plot(x, y, 'k', 'LineWidth', 1.2);
        axis off;


        exportgraphics(gcf, outPath, 'BackgroundColor', 'white',
'ContentType', 'image');
        close(gcf);


        fprintf('[SAVED] %s\n', outName);
    end
end
```

Contrast

```
filePrefix = '20231027';
fileNumbers = 1:21;
interval_length_sec = 15;
num_intervals = 6;
fps = 1550;  %
duration_in_sec = 90;
Nframes = duration_in_sec * fps;
f_sample = 1550;
low_stop = 0.02;
high_stop = 3;
d_st = fdesign.bandpass('N,F3dB1,F3dB2', 10, low_stop, high_stop,
f_sample);
Hd_st = design(d_st, 'butter');
baseFolder = 'contrastcorrV3_signals';
relaxingFolder = fullfile(baseFolder, 'Relaxing_Data');
payingAttentionFolder = fullfile(baseFolder, 'PayingAttention_Data');
if ~exist(relaxingFolder, 'dir'); mkdir(relaxingFolder); end
if ~exist(payingAttentionFolder, 'dir'); mkdir(payingAttentionFolder);
end
% Filter parameters




% === Process Each File ===
for i = 1:numel(fileNumbers)
    fileNum = fileNumbers(i);
```

```
    fileName = sprintf('%s_%d.hdf5', filePrefix, fileNum);
    if ~isfile(fileName); continue; end


    fps = (fileNum <= 18) * 1550 + (fileNum > 18) * 1470;
    framesPerInterval = interval_length_sec * fps;


    % Load and reshape data
    frames = h5read(fileName, '/24505982');
    frames = permute(frames, [3, 1, 2]);  % [frames, height, width]


    % Determine volunteer/trial
    vID = ceil(fileNum / 3);
    tID = mod(fileNum - 1, 3) + 1;


    % Loop over intervals
    for intvl = 1:num_intervals
        % Set label and file path
        if mod(intvl, 2)
            label = 'Relaxing';
            idx = (intvl + 1) / 2;
            outFolder = relaxingFolder;
        else
            label = 'PayingAttention';
            idx = intvl / 2;
            outFolder = payingAttentionFolder;
        end


        outName = sprintf('Volunteer%d_Trial_%d_%s%d.png', vID, tID,
label, idx);
        outPath = fullfile(outFolder, outName);




        % Get interval segment
        startIdx = (intvl - 1) * framesPerInterval + 1;
        endIdx = intvl * framesPerInterval;
        if endIdx > size(frames, 1); continue; end


        segment = frames(startIdx:endIdx, :, :);
        Nframes = size(segment, 1);
        contrastCorr = zeros(1, Nframes - 1);


        %contrast
```

```
        for k = 2:Nframes
            f1 = double(squeeze(segment(k - 1, :, :)));

            f2 = double(squeeze(segment(k, :, :)));


            f1n = (f1 - mean(f1(:))) / std(f1(:));
            f2n = (f2 - mean(f2(:))) / std(f2(:));


            contrastCorr(k - 1) = mean(f1n(:) .* f2n(:));

        % Apply bandpass filter
          contrastCorr_filtered = filter(Hd_st, contrastCorr);
        contrastCorr_filtered = zscore(contrastCorr_filtered);


 fig = figure('Visible', 'off', 'Position', [0, 0, 224, 224]);
        plot(contrastCorr_filtered, 'Color', [0 0.4470 0.7410],
'LineWidth', 1.5);

        axis off;
        set(gca, 'Position', [0 0 1 1]);
        set(gcf, 'Color', 'w');


        % Save
        frame = getframe(gca);
        img = frame2im(frame);
        img_rgb = imresize(img, [224 224]);  % Resize for CNN input
        imwrite(img_rgb, outPath);
        close(fig);


        fprintf('[SAVED] %s (lineplot RGB)\n', outName);
    end
end
```

VGG19

As images were saved only thing differing between codes of different processing techniques is the directory. Contrast plots is used as the example below.

```
outputFolder = 'contrastcorrV3_signals'
customReadFcn = @(filename) imresize(imread(filename), [224 224]);

imds = imageDatastore(outputFolder, 'IncludeSubfolders', true,
'LabelSource', 'foldernames', 'ReadFcn', customReadFcn);
countEachLabel(imds)
```

```
[imdsTrain, imdsValidation] = splitEachLabel(imds, 0.8, 'randomized');
```

```
net = vgg19();
lgraph = layerGraph(net);



numClasses = numel(categories(imdsTrain.Labels));


newLayers = [
    fullyConnectedLayer(numClasses, 'Name', 'fc8_custom', ...
        'WeightLearnRateFactor', 10, 'BiasLearnRateFactor', 10)
    softmaxLayer('Name', 'softmax')
    classificationLayer('Name', 'output')];
```

```
lgraph = replaceLayer(lgraph, 'fc8', newLayers(1));
lgraph = replaceLayer(lgraph, 'prob', newLayers(2));
lgraph = replaceLayer(lgraph, 'output', newLayers(3));
```

```
options = trainingOptions('adam', ...
    'InitialLearnRate', 1e-4, ...
    'MaxEpochs', 30, ...
    'MiniBatchSize', 8, ...
    'Shuffle', 'every-epoch', ...
    'ValidationData', imdsValidation, ...
    'ValidationFrequency', 10, ...
    'Verbose', true, ...
    'Plots', 'training-progress', ...
    'L2Regularization', 1e-4);
```

```
trainedNet = trainNetwork(imdsTrain, lgraph, options);
```

```
predictedLabels = classify(trainedNet, imdsValidation);
trueLabels = imdsValidation.Labels;
accuracy = sum(predictedLabels == trueLabels) / numel(trueLabels);
disp(['Validation Accuracy: ' num2str(accuracy * 100, '%.2f') '%']);


predicted_labels = classify(trainedNet, imdsValidation);
true_labels = imdsValidation.Labels;


predicted_labels = double(predicted_labels);
```

```
true_labels = double(true_labels);
accuracy = sum(predicted_labels == true_labels) / numel(true_labels);
disp(['VGG19 Test Accuracy : ' num2str(accuracy * 100, '%.4f') '%']);


C = confusionmat(true_labels, predicted_labels);
totalsamples = sum(C(:));
percentages = (C / totalsamples) * 100;


figure;
h = heatmap(percentages, ...
    'Colormap', jet, ...
    'ColorLimits', [0, 100], ...
    'FontSize', 10, ...
    'XDisplayLabels', {'0', '1'}, ...
    'YDisplayLabels', {'0', '1'});
title('Contrast - VGG19');
xlabel('Predicted Labels');
ylabel('True Labels');


fprintf('Total number of test samples: %d\n', totalsamples);
```

ResNet-18

As images were saved only thing differing between code of different processing techniques is the directory. Optical flow plots is used as the example below.

```
outputFolder = 'optical_flow_plots'
customReadFcn = @(filename) imresize(imread(filename), [224 224]);

imds = imageDatastore(outputFolder, 'IncludeSubfolders', true,
'LabelSource', 'foldernames', 'ReadFcn', customReadFcn);
countEachLabel(imds)
[imdsTrain, imdsValidation] = splitEachLabel(imds, 0.8, 'randomized');
```

```
net = resnet18();
lgraph = layerGraph(net);
numClasses = numel(categories(imdsTrain.Labels));



newLayers = [
```

```
    fullyConnectedLayer(numClasses, 'Name', 'fc_new',
'WeightLearnRateFactor', 10, 'BiasLearnRateFactor', 10)
    softmaxLayer('Name', 'softmax')
    classificationLayer('Name', 'output')];


lgraph = replaceLayer(lgraph, 'fc1000', newLayers(1));
lgraph = replaceLayer(lgraph, 'prob', newLayers(2));
lgraph = replaceLayer(lgraph, 'ClassificationLayer_predictions',
newLayers(3));
```

```
options = trainingOptions('adam', ...
    'InitialLearnRate', 1e-4, ...
    'MaxEpochs', 10, ...
    'MiniBatchSize', 16, ...
    'Shuffle', 'every-epoch', ...
    'ValidationData', imdsValidation, ...
    'ValidationFrequency', 30, ...
    'Verbose', true, ...
    'Plots', 'training-progress');
```

```
trainedNet = trainNetwork(imdsTrain, lgraph, options);
```

```
predictedLabels = classify(trainedNet, imdsValidation);
trueLabels = imdsValidation.Labels;
accuracy = sum(predictedLabels == trueLabels) / numel(trueLabels);
disp(['Validation Accuracy: ' num2str(accuracy * 100, '%.2f') '%']);


predicted_labels = classify(trainedNet, imdsValidation);
true_labels = imdsValidation.Labels;
predicted_labels = double(predicted_labels);
true_labels = double(true_labels);
accuracy = sum(predicted_labels == true_labels) / numel(true_labels);
disp(['ResNet-18 Test Accuracy : ' num2str(accuracy * 100, '%.4f') '%']);
C = confusionmat(true_labels, predicted_labels);
totalsamples = sum(C(:));
percentages = (C / totalsamples) * 100;
figure;
h = heatmap(percentages, ...
    'Colormap', jet, ...
    'ColorLimits', [0, 100], ...
    'FontSize', 10, ...
    'XDisplayLabels', {'0', '1'}, ...
    'YDisplayLabels', {'0', '1'});
```

```
title('OpticalFlow  - ResNet18');
xlabel('Predicted Labels');
ylabel('True Labels');


fprintf('Total number of test samples: %d\n', totalsamples);
```